

Neural Network Mechanics

Contents

Notation (list of symbols)	3
1 What this chapter covers	5
2 The neuron	6
2.1 The formula	6
2.2 One neuron, computed by hand	6
2.3 What the neuron is: a learned detector	7
2.4 Already familiar, twice	7
2.5 From one neuron to a layer	8
2.6 Symbols	8
3 Activation functions	9
3.1 Why depth needs a nonlinearity	9
3.2 The standard four	9
3.3 Their derivatives: what backpropagation will need	10
3.4 Choosing in practice	11
3.5 Symbols	11
4 The forward pass	12
4.1 Two layers, chained	12
4.2 The running network, by hand	12
4.3 Ending at the loss	14
4.4 Shapes at every stage	14
4.5 A whole batch in one multiplication	15
4.6 Symbols	16
5 Backpropagation	17
5.1 The problem: nine derivatives	17
5.2 The error at the output	18
5.3 The output layer's parameters	18
5.4 Back through the hidden layer	19
5.5 All nine, and the general recipe	21
5.6 One step downhill, as a check	22
5.7 Symbols	24
6 The computational graph	25
6.1 From computation to graph	25
6.2 When paths branch, gradients add	25
6.3 Autograd: the machinery in software	26
6.4 Symbols	27
7 Weight initialization	28
7.1 Failure one: symmetry never breaks itself	28
7.2 Failure two: the wrong scale compounds with depth	29
7.3 The fix: random, sized to the fan-in	30
7.4 Symbols	30

8	Vanishing and exploding gradients	31
8.1	The gradient is a product with D factors	31
8.2	Vanishing: the sigmoid’s quarter ceiling	31
8.3	Exploding: the same product, upward	32
8.4	The remedies, most already met	32
8.5	Symbols	33
9	Batches and epochs	34
9.1	One step = one batch	34
9.2	The epoch, and the shuffle	35
9.3	What sets B in practice	35
9.4	Symbols	36
10	Optimizers	37
10.1	Two complaints about the plain rule	37
10.2	Momentum: remember the direction	37
10.3	Adam: a step size per parameter	38
10.4	Choosing in practice	39
10.5	Symbols	39
11	Learning-rate schedules and warmup	40
11.1	Why one constant η cannot win	40
11.2	Warmup: small steps while nothing is trustworthy	40
11.3	Decay: settle into the minimum	40
11.4	Symbols	41
12	Gradient clipping	42
12.1	The rule	42
12.2	Where it sits, and why it still matters	42
12.3	In practice	43
12.4	Symbols	43
13	Normalization	44
13.1	The operation: standardize, then re-dress	44
13.2	BatchNorm and LayerNorm: which direction you average	44
13.3	Where it sits in the network	47
13.4	Symbols	47
14	Dropout	48
14.1	The rule	48
14.2	Why silencing helps	48
14.3	Symbols	49
15	Parameters and hyperparameters	50
15.1	The inventory	50
15.2	How hyperparameters get chosen	50
15.3	Symbols	51

Notation (list of symbols)

w	a neuron's weights: one learned number per input, collected in a vector.
b	a neuron's bias: a learned offset added to the weighted sum.
$\hat{y}$	the network's prediction: the output layer's activation.
y	the true label carried by a training input; what the prediction is scored against.
L	the loss: the single number scoring the prediction against the label.
$\delta^{(\ell)}$	layer ℓ 's error: the gradient of the loss with respect to that layer's pre-activation, $\partial L / \partial z^{(\ell)}$.

Convention — indexing. Unless a section says otherwise, indices are *1-based*: a vector $x \in \mathbb{R}^n$ has entries $x_1, \dots, x_n$, and matrix rows and columns count from 1. Where 0-based indexing is the natural choice (token positions, anything mirroring code), the section says so explicitly.

1 What this chapter covers

How a neural network actually computes and actually learns: the forward pass that turns input into prediction, and backpropagation — the chain rule walked backward — that turns the loss into a gradient for every parameter at once.

The concepts (★ = load-bearing):

- **The neuron** — weighted sum plus bias, passed through a nonlinearity: $\text{activation}(w \cdot x + b)$. (§2)
- **Activation functions ★** — ReLU, sigmoid, tanh, softmax; without a nonlinearity, stacked linear layers collapse into one linear layer — depth would add nothing. (§3)
- **Forward pass** — input $\rightarrow$ layer 1 $\rightarrow \dots \rightarrow$ prediction $\rightarrow$ loss; each layer is a matrix multiplication plus an activation. (§4)
- **Backpropagation ★** — the chain rule walked *backward* through the computational graph, giving the gradient of the loss w.r.t. every parameter in one sweep. *The answer to “how do models learn?”*. (§5)
- **Computational graph** — the network as a graph of operations; autograd records the forward pass and runs backprop automatically. (§6)
- **Weight initialization** — bad starting values make signals explode or die; Xavier/He schemes keep variance stable. (§7)
- **Vanishing / exploding gradients** — gradients multiply layer after layer; why deep nets were historically untrainable, and why ReLU, normalization, and residual connections exist. (§8)
- **Batches & epochs** — batch = examples per step; epoch = one full pass over the data. (§9)
- **Optimizers** — momentum (smooths the path), Adam (per-parameter step sizes; the default). (§10)
- **Learning-rate schedules & warmup** — the learning rate is varied over training: warm up, then decay; standard in all large-model training. (§11)
- **Gradient clipping** — cap the gradient’s size before stepping; a guard against exploding gradients. (§12)
- **Normalization (BatchNorm / LayerNorm)** — rescale activations to stable ranges; LayerNorm is what transformers use. (§13)
- **Dropout** — randomly silence neurons during training; a neural-net-specific regularizer. (§14)
- **Parameters vs. hyperparameters** — learned by training vs. chosen by you. (§15)

2 The neuron

Every neural network, however large, is assembled from one small unit repeated millions of times. This section builds that unit: the *neuron* — a weighted sum plus a bias, passed through a nonlinearity — computes one by hand, and shows what it does: it is a learned yes/no detector, and it is the direct generalization of the one-parameter model that *What Learning Means* trained.

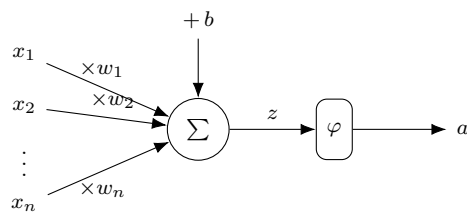
2.1 The formula

A *neuron* is a function that takes a vector of inputs and returns a single number: the weighted sum of the inputs, plus an offset, passed through a fixed nonlinear function. Written out, with every part named:

$$a = \varphi\left(\underbrace{w \cdot x}_{\text{weighted sum}} + \underbrace{b}_{\text{bias}}\right),$$

- $x \in \mathbb{R}^n$ is the input: n features, as a vector.
- $w \in \mathbb{R}^n$ holds the *weights*: one learned number per input. The dot product $w \cdot x = \sum_i w_i x_i$ (*Essential Math for AI*) multiplies each feature by its weight and adds.
- $b \in \mathbb{R}$ is the *bias*: a learned offset added to the sum — a single scalar, not a vector.
- φ is the *activation function*: a fixed nonlinearity chosen by the person, not learned. The next section is devoted to them; today one concrete choice suffices — the sigmoid $\sigma(z) = 1/(1 + e^{-z})$, which squashes any real number into $(0, 1)$.

The intermediate value $z = w \cdot x + b$ — the number *before* the activation — is called the *pre-activation*; the output a is the *activation*. As a picture:



The learned parameters are w and b — $n + 1$ numbers per neuron. They are exactly a θ in the sense of *What Learning Means*: values the update rule adjusts.

2.2 One neuron, computed by hand

Take $n = 2$, the input $x = (2, 1)^\top$, and a neuron with $w = (0.5, -1)^\top$, $b = 1$, $\varphi = \sigma$. Run the formula:

$$\begin{aligned} z &= w \cdot x + b && \text{(the pre-activation)} \\ &= 0.5 \cdot 2 + (-1) \cdot 1 + 1 && \text{(insert } w, x, b) \\ &= 1 - 1 + 1 = 1, && \text{(multiply, add)} \\ a &= \sigma(1) = \frac{1}{1 + e^{-1}} = \frac{1}{1 + 0.368} \approx 0.731. && \text{(apply the activation)} \end{aligned}$$

The neuron answers 0.731 — a soft “yes”. Feed it a different input, $x = (1, 3)^\top$:

$$z = 0.5 \cdot 1 + (-1) \cdot 3 + 1 = -1.5, \quad a = \sigma(-1.5) \approx 0.182$$

— a soft “no”. Same neuron, opposite verdicts: the weights decide which inputs it says yes to.

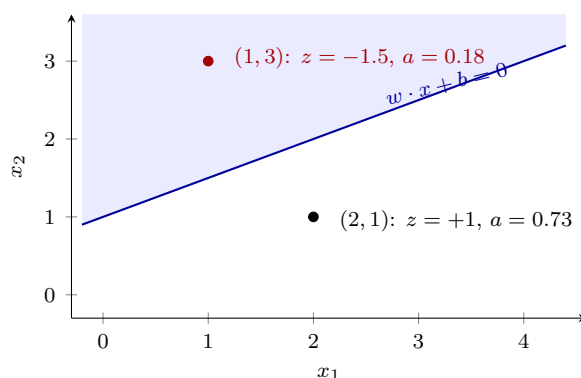
2.3 What the neuron is: a learned detector

Each part of the formula has a job, and they compose into one behavior.

The dot product scores a match. *Essential Math for AI* read $w \cdot x$ as alignment: it is large when x points the way w points. So the weight vector is a *pattern*, and the sum scores how strongly the input matches it. The weights here, $(0.5, -1)^\top$, describe a pattern that likes feature 1 and dislikes feature 2.

The bias moves the threshold. Without b , the boundary between yes and no would be forced through $z = w \cdot x = 0$. The bias shifts it: $b > 0$ makes the neuron easier to please, $b < 0$ stricter. It is the neuron’s adjustable default opinion.

The activation turns the score into a decision. The sigmoid maps the unbounded score z into $(0, 1)$: scores far above 0 approach a firm yes (1), far below a firm no (0), and $z = 0$ gives exactly $\sigma(0) = 0.5$ — undecided. The dividing set $w \cdot x + b = 0$ is where the neuron switches sides. With two inputs it is a *line* in the input plane (in higher dimension, a plane; in general a *hyperplane* — the flat $n-1$ -dimensional analogue). For our neuron the line is $0.5x_1 - x_2 + 1 = 0$, i.e. $x_2 = 0.5x_1 + 1$:



the neuron’s decision boundary $0.5x_1 - x_2 + 1 = 0$; below the line $z > 0$ (yes side), in the shaded region $z < 0$ (no side)

One neuron, then, can only draw one straight boundary — it separates the input space into two half-spaces and nothing more. Everything a deep network adds — activations, layers, depth — exists to bend and combine these straight cuts into arbitrary shapes.

2.4 Already familiar, twice

The neuron is not a new kind of object — the previous chapters trained two of them without the name.

The running model. *What Learning Means* trained $\hat{y} = \theta x$: one input, weight θ , no bias, no activation. That is a neuron stripped to its skeleton — and everything that chapter established applies verbatim to the full neuron, because w and b are ordinary parameters: a loss scores the neuron’s answers, gradient descent walks (w, b) downhill, batches estimate the gradient, regularization shrinks the weights.

Logistic regression. A single sigmoid neuron whose output a is read as a probability, trained with the cross-entropy loss, is exactly the classical model called *logistic regression* — for decades the workhorse of yes/no prediction. A neural network begins where one such unit ends.

2.5 From one neuron to a layer

Networks never use one neuron; they use them in groups. A *layer* is m neurons reading the *same* input, each with its own weights and bias. Stack the m weight vectors as the rows of a matrix $W \in \mathbb{R}^{m \times n}$, the biases into $b \in \mathbb{R}^m$, and the whole layer is one line:

$$a = \varphi(Wx + b), \quad \underbrace{W}_{m \times n} \underbrace{x}_{n \times 1} + \underbrace{b}_{m \times 1} \longrightarrow \underbrace{a}_{m \times 1}$$

with φ applied to each entry separately. This is precisely *Essential Math for AI*'s reading of $y = Wx$ — each row of a weight matrix is one feature detector — now with its bias and nonlinearity attached. How layers chain into a network is the forward pass, two sections ahead; first, the next section examines the piece whose absence would collapse everything: the activation function.

2.6 Symbols

New persistent symbols: w and b — see the [list of symbols](#). Local to this section:

Local symbol	Meaning (this section)
φ	the activation function: a fixed nonlinearity (next section)
$z = w \cdot x + b$	the pre-activation: the score before the nonlinearity
$a = \varphi(z)$	the activation: the neuron's output
$\sigma(z)$	the sigmoid $1/(1 + e^{-z})$, squashing scores into $(0, 1)$
hyperplane	the flat boundary $w \cdot x + b = 0$; a line when $n = 2$
W, m	a layer's weight matrix (one row per neuron); its neuron count

3 Activation functions

§2 passed the score z through a nonlinearity φ without asking why. This section asks why — and proves the answer: without φ , a network of any depth collapses into a single straight-line model, so depth would buy nothing. It then introduces the four standard activation functions, computes them on §2.2’s scores, and records their derivatives — the numbers backpropagation will need.

3.1 Why depth needs a nonlinearity

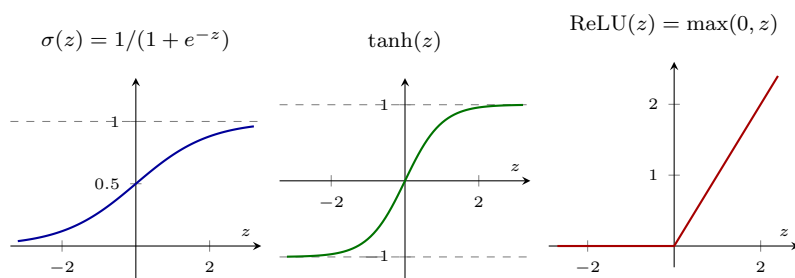
An *activation function* is a fixed nonlinear function applied to every neuron’s pre-activation; it is chosen by the person, has no learned parameters, and acts on each number separately. To see why it cannot be dropped, drop it. Stack two linear layers — §2.5’s $a = Wx + b$ with no φ — and multiply out:

$$\begin{aligned} a &= W_2 (W_1 x + b_1) + b_2 && \text{(layer 1 feeds layer 2)} \\ &= W_2 W_1 x + W_2 b_1 + b_2 && \text{(multiply out)} \\ &= W' x + b', \quad W' = W_2 W_1, \quad b' = W_2 b_1 + b_2 && \text{(name the products)} \end{aligned}$$

— exactly one linear layer, with weight matrix W' and bias b' . The same collapse repeats for any depth: each extra linear layer multiplies into the same single matrix. And a single linear layer, §2.3 showed, can only draw straight boundaries. So a deep network without activation functions equals a shallow straight-line model — millions of parameters, one straight cut. The nonlinearity between layers is what stops the collapse; it is the only reason depth exists.

3.2 The standard four

Here are the three elementwise functions, each on its own axes; the dashed lines mark the values the curves approach but never reach:



Sigmoid. $\sigma(z) = 1/(1 + e^{-z})$, already used in §2.1: it squashes any score into $(0, 1)$, so its output reads as a probability — the natural choice when the answer is yes/no.

Tanh. The *hyperbolic tangent*,

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}},$$

squashes any score into $(-1, 1)$. It is really a recentered sigmoid, $\tanh(z) = 2\sigma(2z) - 1$, and the

chain from the definition shows it:

$$\begin{aligned}
 \tanh(z) &= \frac{e^z - e^{-z}}{e^z + e^{-z}} && \text{(the definition)} \\
 &= \frac{1 - e^{-2z}}{1 + e^{-2z}} && \text{(multiply top and bottom by } e^{-z}) \\
 &= \frac{2 - (1 + e^{-2z})}{1 + e^{-2z}} && \text{(write the top } 1 - e^{-2z} \text{ as } 2 - (1 + e^{-2z})) \\
 &= \frac{2}{1 + e^{-2z}} - 1 && \text{(split the fraction)} \\
 &= 2\sigma(2z) - 1. && \text{(recognize } \sigma(2z) = 1/(1 + e^{-2z}))
 \end{aligned}$$

Its advantage over σ inside a network: outputs centered on 0 rather than 0.5, which keeps the next layer's inputs balanced.

ReLU. The *rectified linear unit*,

$$\text{ReLU}(z) = \max(0, z)$$

— negative scores become 0, positive scores pass through unchanged. It barely looks nonlinear, but the kink at 0 is nonlinearity enough to stop the collapse, and it is the modern default for every hidden layer: costs one comparison, and (next subsection) its derivative does not fade.

Softmax. The odd one out: it takes a whole *vector* of scores $z = (z_1, \dots, z_K)^\top$ — here z holds K scores, one per class, not the single scalar of the paragraphs above — and returns a probability distribution over them (*Essential Math for AI* built it):

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K$$

— entry k exponentiates its own score and divides by the sum of all the exponentials, so the K outputs are positive and add up to 1. It is not applied entry by entry — through the shared denominator every output depends on all the scores — and it appears in one place only: the output layer of a classifier.

On §2.2's two pre-activations:

z	$\sigma(z)$	$\tanh(z)$	$\text{ReLU}(z)$
1	0.731	0.762	1
−1.5	0.182	−0.905	0

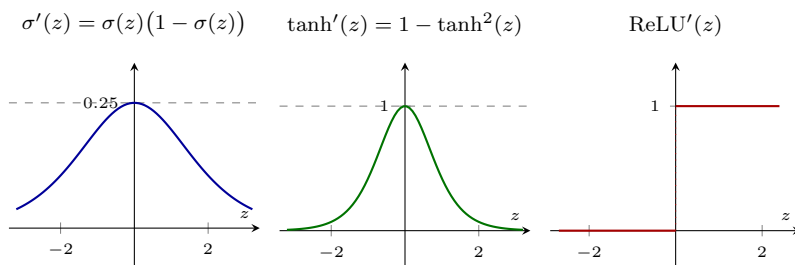
3.3 Their derivatives: what backpropagation will need

Training differentiates everything on the path from parameters to loss, and the activation sits on that path — so each φ contributes its derivative as one factor of the chain rule. The three elementwise activations:

$\varphi(z)$	$\varphi'(z)$	Largest value	Where it is nearly 0
$\sigma(z)$	$\sigma(z)(1 - \sigma(z))$	0.25 (at $z = 0$)	both tails ($ z $ large)
$\tanh(z)$	$1 - \tanh^2(z)$	1 (at $z = 0$)	both tails
$\text{ReLU}(z)$	0 for $z < 0$, 1 for $z > 0$	1	the whole negative side

($\sigma' = \sigma(1 - \sigma)$) was derived in *Essential Math for AI*; ReLU’s kink at $z = 0$ has no derivative there — software treats it as 0, the same convention as the L1 penalty’s kink in *What Learning Means*.)

Here are the three derivatives, each on its own axes; the dashed lines mark the largest values of the two smooth curves, and ReLU’ jumps from 0 to 1 at the kink:



The table’s last two columns preview a battle two sections ahead. When $|z|$ is large, σ and $\tanh$ *saturate*: the curve flattens, the derivative drops toward 0, and a gradient passing through the layer is multiplied by nearly nothing — stack many such layers and the gradient *vanishes*. ReLU’s derivative is exactly 1 on the whole positive side: a gradient passes through undiminished. This single property is a large part of why deep networks became trainable. The price is ReLU’s own failure mode: a neuron whose score stays negative outputs 0 and receives gradient 0 — it can get stuck, permanently silent (a *dead neuron*); the vanishing-gradients section returns to both problems.

3.4 Choosing in practice

The choice is nearly mechanical. *Hidden layers*: ReLU, unless there is a specific reason otherwise. *Output layer*: match the task — a sigmoid for one yes/no probability, a softmax for a distribution over K classes, and *no activation at all* for regression, where the answer is an unbounded number. The output activation is dictated by what the loss expects; the hidden activations are dictated by trainability.

3.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
φ, φ'	an activation function and its derivative
$\tanh(z)$	the recentered sigmoid: squashes into $(-1, 1)$
$\text{ReLU}(z)$	$\max(0, z)$: zero below 0, identity above
saturation	the flat tails of $\sigma/\tanh$, where $\varphi' \approx 0$
dead neuron	a ReLU neuron stuck on the negative side: output 0, gradient 0
W', b'	the collapsed single layer: $W' = W_2 W_1$, $b' = W_2 b_1 + b_2$

4 The forward pass

§2.5 built a single layer, and §3.4 chose its nonlinearity; what is still missing is how layers combine into a network. This section supplies it. The *forward pass* is the front-to-back run of a network: the input enters, every layer transforms it in turn, a prediction $\hat{y}$ comes out, and the loss scores that prediction. We wire a two-layer network out of the pieces already built, run it by hand on §2.2's inputs — these numbers become the chapter's running network, the one backpropagation will differentiate — record the shape of every intermediate, and close by pushing a whole batch through in one matrix multiplication.

4.1 Two layers, chained

A *feedforward network* is a function built by chaining layers: the output vector of each layer is the input vector of the next. The smallest case that is more than one layer has two:

$$h = \varphi(W_1 x + b_1), \quad \hat{y} = \psi(W_2 h + b_2).$$

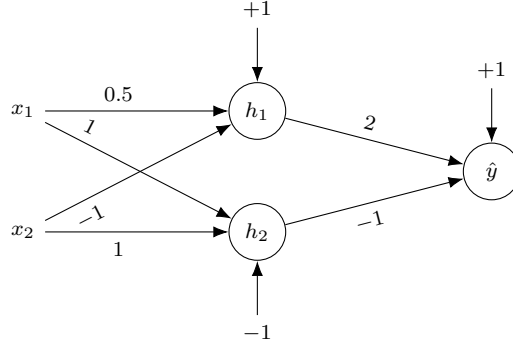
- The first layer is a *hidden layer*: a layer whose output is not the network's answer, only internal material — it sits hidden between input and output, hence the name. Its output vector is written h .
- The second layer is the *output layer*: the last layer; its output is the network's prediction $\hat{y}$.
- The two activations follow §3.4's rules: the hidden φ is ReLU, and the output activation — written ψ because it need not equal φ — is dictated by the task. This section predicts a yes/no probability, so $\psi = \sigma$.
- Notation: subscripts number the layers' parameters (W_1, b_1 belong to layer 1, as in §3.1's collapse proof). A pre-activation needs its subscript slot for components, so the layer label moves up, in parentheses: $z^{(1)} = W_1 x + b_1$ with components $z^{(1)} = (z_1^{(1)}, z_2^{(1)})^\top$, and $z^{(2)} = W_2 h + b_2$. The (1) is a layer label, not a power.
- Counting convention: the input is not counted as a layer — it computes nothing. Some books draw the input as a column of nodes and call this same picture a three-layer network.

4.2 The running network, by hand

Here is the concrete network the rest of the chapter keeps: two inputs, two hidden neurons, one output neuron,

$$W_1 = \begin{pmatrix} 0.5 & -1 \\ 1 & 1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \quad \varphi = \text{ReLU}; \quad W_2 = \begin{pmatrix} 2 & -1 \end{pmatrix}, \quad b_2 = 1, \quad \psi = \sigma.$$

Row 1 of W_1 together with the first entry of b_1 is exactly §2.2's neuron — weights $(0.5, -1)^\top$, bias 1 — now working as hidden neuron 1, its activation switched to ReLU per the hidden-layer rule.



each edge is labeled with its weight; the vertical arrow into each circle carries the bias. Circles h_1 , h_2 each perform one full neuron of §2.1 (weighted sum plus bias, then ReLU); the $\hat{y}$ circle applies the sigmoid

Run the network on §2.2's first input, $x = (2, 1)^\top$. Layer 1:

$$\begin{aligned}
 z^{(1)} &= W_1 x + b_1 && \text{(the hidden pre-activations)} \\
 &= \begin{pmatrix} 0.5 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \end{pmatrix} && \text{(insert } W_1, x, b_1) \\
 &= \begin{pmatrix} 0.5 \cdot 2 + (-1) \cdot 1 \\ 1 \cdot 2 + 1 \cdot 1 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \end{pmatrix} && \text{(each row of } W_1 \text{ dots } x) \\
 &= \begin{pmatrix} 0 \\ 3 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, && \text{(multiply; then add the biases)} \\
 h = \text{ReLU}(z^{(1)}) &= \begin{pmatrix} \text{ReLU}(1) \\ \text{ReLU}(2) \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}. && \text{(both positive: ReLU passes both)}
 \end{aligned}$$

Hidden neuron 1 scored $z_1^{(1)} = 1$ — the same 1 that §2.2 computed by hand for this input. Layer 2 reads h , not x :

$$\begin{aligned}
 z^{(2)} &= W_2 h + b_2 && \text{(the output pre-activation)} \\
 &= 2 \cdot 1 + (-1) \cdot 2 + 1 && \text{(insert } W_2, h, b_2) \\
 &= 1, && \text{(multiply, add)} \\
 \hat{y} &= \sigma(1) \approx 0.731. && \text{ (§3.2's values table)}
 \end{aligned}$$

The network answers 0.731 — a soft yes. In the detector language of §2.3: hidden neuron 1 is the old detector that likes feature 1 and dislikes feature 2; hidden neuron 2, weights $(1, 1)^\top$, likes both features; the output layer trusts detector 1 (weight 2), counts detector 2 as evidence for no (weight -1), and leans toward yes by default ($b_2 = 1$).

The same network on §2.2's other input, $x = (1, 3)^\top$, one line per stage:

$$\begin{aligned}
 z^{(1)} &= \begin{pmatrix} 0.5 \cdot 1 + (-1) \cdot 3 + 1 \\ 1 \cdot 1 + 1 \cdot 3 + (-1) \end{pmatrix} = \begin{pmatrix} -1.5 \\ 3 \end{pmatrix}, && h = \begin{pmatrix} 0 \\ 3 \end{pmatrix}, \\
 z^{(2)} &= 2 \cdot 0 + (-1) \cdot 3 + 1 = -2, && \hat{y} = \sigma(-2) \approx 0.119.
 \end{aligned}$$

This time ReLU actually acts: hidden neuron 1's score is -1.5 — again §2.2's number for this input — and ReLU zeroes it, silencing the detector that did not match. The network answers 0.119: a fairly firm no.

Keep the intermediates $z^{(1)}, h, z^{(2)}$ in sight. The next section walks this same path backward, and it will need every one of them.

4.3 Ending at the loss

A forward pass ends one step after the prediction: training needs the prediction *scored*. Each training input carries its true *label* y — the answer the network should have given; here $y = 1$, meaning the true answer for $(2, 1)^\top$ is yes. For a yes/no task with a sigmoid output, the standard score is the *cross-entropy loss*. *Essential Math for AI* built the cross-entropy of two distributions p and q — the average surprise when outcomes actually follow p but every surprise is computed from the model’s probabilities q :

$$H(p, q) = - \sum_k p_k \ln q_k.$$

Here the outcome has exactly two values, yes and no, and both distributions are in hand. The truth puts probability y on yes and $1 - y$ on no — with $y = 1$, everything on yes — so $p = (y, 1 - y)$. The network puts $\hat{y}$ on yes, so $q = (\hat{y}, 1 - \hat{y})$. The loss is the cross-entropy of this pair; writing the sum out gives its usual two-term form:

$$\begin{aligned} L &= - \sum_{k \in \{\text{yes}, \text{no}\}} p_k \ln q_k && \text{(cross-entropy, two outcomes)} \\ &= - [p_{\text{yes}} \ln q_{\text{yes}} + p_{\text{no}} \ln q_{\text{no}}] && \text{(write the sum out)} \\ &= - [y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]. && \text{(insert } p = (y, 1 - y), q = (\hat{y}, 1 - \hat{y})) \end{aligned}$$

Populate this with the run just finished, $\hat{y} = 0.731$ and $y = 1$:

$$\begin{aligned} L &= - [y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})] && \text{(the two-term form just derived)} \\ &= - [1 \cdot \ln 0.731 + (1 - 1) \cdot \ln(1 - 0.731)] && \text{(insert } y = 1, \hat{y} = 0.731) \\ &= - \ln 0.731 && \text{(the second term vanishes: } 1 - 1 = 0) \\ &\approx 0.313. && \text{(evaluate: } \ln 0.731 \approx -0.313) \end{aligned}$$

The number reads as the price of doubt about the truth: a prediction of 0.5 would have cost $-\ln 0.5 \approx 0.693$, a prediction of 0.99 only $-\ln 0.99 \approx 0.010$. One forward pass therefore delivers one scored example: input $(2, 1)^\top$, prediction 0.731, truth 1, loss 0.313. The gradient descent of *What Learning Means* now wants the derivative of this 0.313 with respect to every weight and bias — that computation is the next section.

4.4 Shapes at every stage

Every object in the run has a definite shape, and recording them once prevents most implementation errors:

Object	Shape	In the example	What it is
x	$n \times 1$	2×1	the input
W_1	$m \times n$	2×2	hidden weights: one row per hidden neuron
b_1	$m \times 1$	2×1	hidden biases
$z^{(1)}, h$	$m \times 1$	2×1	hidden pre-activations; hidden activations
W_2	$1 \times m$	1×2	output weights
$b_2, z^{(2)}, \hat{y}$	1×1	1×1	the output layer’s scalars
y, L	1×1	1×1	the label; the loss

The pattern generalizes. A layer that turns n_{in} numbers into n_{out} numbers needs W of shape $n_{\text{out}} \times n_{\text{in}}$ and b of shape $n_{\text{out}} \times 1$; two layers can be chained exactly when the second layer's n_{in} equals the first layer's n_{out} . A network of any depth is the same step repeated:

$$h^{(0)} = x, \quad h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell), \quad \ell = 1, \dots, D, \quad \hat{y} = h^{(D)},$$

with the last layer using ψ instead of φ . (The number of layers is written D , not L — this chapter reserves L for the loss.) And however large the network, the loss at the end is a single number — one scalar for gradient descent to push downhill, exactly as in *What Learning Means*.

4.5 A whole batch in one multiplication

What Learning Means trains on a batch of B examples per step, so every step needs B forward passes. Running them one by one works; packing them into one matrix multiplication is how it is actually done. Stack the batch's inputs as the *rows* of a matrix:

$$X = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix} \in \mathbb{R}^{B \times n}, \quad B = 2$$

— row i holds example i ; this batch is exactly the two inputs already run. The layer formulas were written for a column vector x , so with the examples lying in rows the weights multiply from the right, transposed (*Essential Math for AI*: the transpose swaps rows and columns):

$$Z^{(1)} = XW_1^\top + b_1^\top, \quad H = \text{ReLU}(Z^{(1)}),$$

where adding the row vector $b_1^\top$ to a matrix means adding it to *every* row — the convention called *broadcasting*. By hand:

$$\begin{aligned} XW_1^\top &= \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 0.5 & 1 \\ -1 & 1 \end{pmatrix} && (W_1^\top: W_1\text{'s rows become columns}) \\ &= \begin{pmatrix} 2 \cdot 0.5 + 1 \cdot (-1) & 2 \cdot 1 + 1 \cdot 1 \\ 1 \cdot 0.5 + 3 \cdot (-1) & 1 \cdot 1 + 3 \cdot 1 \end{pmatrix} && (\text{entry } (i, j): \text{ example } i \text{ dots neuron } j\text{'s weights}) \\ &= \begin{pmatrix} 0 & 3 \\ -2.5 & 4 \end{pmatrix}, && (\text{multiply, add}) \\ Z^{(1)} &= \begin{pmatrix} 0 & 3 \\ -2.5 & 4 \end{pmatrix} + \begin{pmatrix} 1 & -1 \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ -1.5 & 3 \end{pmatrix}, && (b_1^\top = (1, -1) \text{ broadcast onto both rows}) \\ H &= \text{ReLU}(Z^{(1)}) = \begin{pmatrix} 1 & 2 \\ 0 & 3 \end{pmatrix}. && (\text{per entry}) \end{aligned}$$

Compare with the single-input runs: row 1 of H is $(1, 2)$ — the hidden activations of input $(2, 1)^\top$ — and row 2 is $(0, 3)$ — those of $(1, 3)^\top$. The batch computed nothing new; it packed B forward passes into one multiplication. The output layer works the same way:

$$\begin{aligned} Z^{(2)} &= HW_2^\top + b_2 = \begin{pmatrix} 1 & 2 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \end{pmatrix} + 1 && (\text{insert } H, W_2^\top, b_2) \\ &= \begin{pmatrix} 1 \cdot 2 + 2 \cdot (-1) \\ 0 \cdot 2 + 3 \cdot (-1) \end{pmatrix} + 1 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, && (\text{multiply; broadcast } b_2 = 1) \\ \hat{Y} &= \sigma(Z^{(2)}) = \begin{pmatrix} 0.731 \\ 0.119 \end{pmatrix}. && (\text{per entry: the two answers again}) \end{aligned}$$

The shapes compose as $(B \times n)(n \times m) \rightarrow B \times m$ for the hidden layer and $(B \times m)(m \times 1) \rightarrow B \times 1$ for the output layer. Each row of $\hat{Y}$ is then scored against its own label, and the batch loss is the average of the B per-example losses — the quantity that the SGD of *What Learning Means* steps on. The reason for all this packing is hardware: one large matrix multiplication runs far faster than B small ones, and this fact — not mathematics — is why the batch is the working unit of practice.

4.6 Symbols

New persistent symbols: $\hat{y}$, y , and L — see the [list of symbols](#). Local to this section:

Local symbol	Meaning (this section)
h	the hidden layer's activation vector
$z^{(\ell)}$	layer ℓ 's pre-activation vector; (ℓ) is a layer label, not a power
W_ℓ, b_ℓ	layer ℓ 's weight matrix and bias vector
ψ	the output layer's activation (σ here)
D	the number of layers (L is reserved for the loss)
X, B	the batch matrix (one example per row); the batch size
$Z^{(\ell)}, H, \hat{Y}$	batch versions of $z^{(\ell)}, h, \hat{y}$: one row per example
broadcasting	adding a row vector to every row of a matrix

5 Backpropagation

§4.3 ended with $L = 0.313$ and a debt: gradient descent needs the derivative of that loss with respect to every one of the network’s nine parameters. *Backpropagation* is the algorithm that computes the gradient of the loss with respect to every parameter by walking the chain rule backward through the network — from the loss, layer by layer, back to the input — reusing the intermediates the forward pass stored, and handing each parameter its gradient on the way past. This section runs it by hand on the running network: nine derivatives in one sweep, one of them re-checked by a nudge, and at the end an actual gradient-descent step that makes the loss fall.

5.1 The problem: nine derivatives

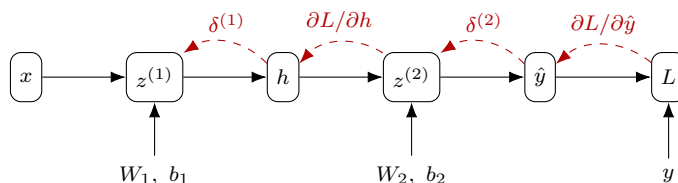
The running network has nine parameters: four in W_1 , two in b_1 , two in W_2 , one in b_2 . The update rule of *What Learning Means* moves every one of them against its own partial derivative, so what is wanted is

$$\frac{\partial L}{\partial \theta} \quad \text{for each of the nine } \theta$$

— $\partial L/\partial \theta$ being *Essential Math for AI*’s partial derivative: how L responds to a small nudge of θ alone. Everything the computation needs was produced and stored by the hand run of §4.2 — the run on the *first* input, $x = (2, 1)^\top$, not the second input and not the batch:

$$x = (2, 1)^\top, \quad y = 1; \quad z^{(1)} = (1, 2)^\top, \quad h = (1, 2)^\top, \quad z^{(2)} = 1, \quad \hat{y} = 0.731, \quad L = 0.313.$$

The tool is the chain rule, and the road map is the dependency chain of the forward pass itself. L is computed from $\hat{y}$, which is computed from $z^{(2)}$, which is computed from W_2, b_2 , and h — and so on back to the input:



solid arrows: the forward pass; dashed arrows: the direction the gradient is computed in. Each parameter collects its gradient at the station where it feeds in

The plan reads off the picture. Start at L and walk left: at each station, compute the gradient of L with respect to that station’s value, using only the station one step to the right — one chain-rule link at a time. Whenever a station has parameters feeding in, their gradients come out as a cheap by-product. By the time the walk reaches the input, every parameter has been served.

5.2 The error at the output

First station: $\hat{y}$. Differentiate the loss of §4.3 with respect to it:

$$\begin{aligned}
 L &= -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})] && \text{(the loss)} \\
 \frac{\partial L}{\partial \hat{y}} &= -\left[\frac{y}{\hat{y}} + (1 - y) \cdot \frac{-1}{1 - \hat{y}}\right] && ((\ln u)' = u'/u; \text{ inside } \ln(1 - \hat{y}) \text{ the inner derivative is } -1) \\
 &= -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} && \text{(distribute the minus sign)} \\
 &= -\frac{1}{0.731} + \frac{1 - 1}{1 - 0.731} && \text{(insert } y = 1, \hat{y} = 0.731) \\
 &= -1.368. && \text{(the second term vanishes: } 1 - 1 = 0)
 \end{aligned}$$

The slope is negative: pushing $\hat{y}$ up would push the loss down. That is common sense in derivative form — the truth is 1, so the network should be more confident in yes.

Next station: $z^{(2)}$. Recall how the two are related: the forward pass computed the prediction *from* this station, $\hat{y} = \sigma(z^{(2)})$, and it stored the value $z^{(2)} = 1$ (§4.2's run on $x = (2, 1)^\top$) — which is why $\hat{y} = \sigma(1) = 0.731$. That is the only road from $z^{(2)}$ to L , so the chain rule multiplies the two links. The result is important enough to get a name: the layer's *error* $\delta^{(\ell)}$, defined as $\delta^{(\ell)} = \partial L / \partial z^{(\ell)}$.

$$\begin{aligned}
 \delta^{(2)} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz^{(2)}} && \text{(chain rule: } z^{(2)} \text{ acts on } L \text{ only through } \hat{y}) \\
 &= \frac{\partial L}{\partial \hat{y}} \cdot \sigma'(z^{(2)}) && (\hat{y} = \sigma(z^{(2)}): \text{ this link's slope is } \sigma') \\
 &= \frac{\partial L}{\partial \hat{y}} \cdot \sigma(z^{(2)}) (1 - \sigma(z^{(2)})) && (\sigma' = \sigma(1 - \sigma), \text{ §3.3}) \\
 &= (-1.368) \cdot \sigma(1) (1 - \sigma(1)) && \text{(insert } \partial L / \partial \hat{y} = -1.368 \text{ and } z^{(2)} = 1) \\
 &= (-1.368) \cdot (0.731 \cdot (1 - 0.731)) && (\sigma(1) = 0.731 \text{ — which is } \hat{y} \text{ itself}) \\
 &= (-1.368) \cdot 0.197 = -0.269. && \text{(multiply)}
 \end{aligned}$$

There is a hidden simplification worth seeing once. Multiply the two links in general form, before any numbers:

$$\begin{aligned}
 \delta^{(2)} &= \left[-\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}}\right] \cdot \hat{y} (1 - \hat{y}) && \text{(the two links, general)} \\
 &= -y (1 - \hat{y}) + (1 - y) \hat{y} && (\hat{y} \text{ cancels in the first term, } 1 - \hat{y} \text{ in the second}) \\
 &= -y + y\hat{y} + \hat{y} - y\hat{y} && \text{(expand both products)} \\
 &= \hat{y} - y. && \text{(the } y\hat{y} \text{ terms cancel)}
 \end{aligned}$$

So the output error is exactly *prediction minus truth*: $\delta^{(2)} = 0.731 - 1 = -0.269$, the same number. The sigmoid-cross-entropy pair is engineered so that this holds — the network's error signal is its signed miss.

5.3 The output layer's parameters

For the by-hand work the entries of the matrices need names, and the convention is the one §4.1 already uses for pre-activations: the layer label rides up in parentheses. So $w_1^{(2)}, w_2^{(2)}$ are the two

entries of W_2 , $w_{ij}^{(1)}$ is the entry in row i , column j of W_1 , and $(b_1)_i$ is entry i of b_1 . Written with entries, the output layer is

$$z^{(2)} = w_1^{(2)}h_1 + w_2^{(2)}h_2 + b_2.$$

Each parameter reaches L along one full path — parameter $\rightarrow z^{(2)} \rightarrow \hat{y} \rightarrow L$ — so the chain rule writes each gradient as a product of three links, and the first two links together are exactly the $\delta^{(2)}$ just computed:

$$\begin{aligned} \frac{\partial L}{\partial w_1^{(2)}} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial w_1^{(2)}} && \text{(chain rule along } w_1^{(2)} \rightarrow z^{(2)} \rightarrow \hat{y} \rightarrow L) \\ &= \delta^{(2)} \cdot \frac{\partial z^{(2)}}{\partial w_1^{(2)}} && \text{(the first two links are exactly } \delta^{(2)}) \\ &= \delta^{(2)} \cdot h_1 && \text{(only } w_1^{(2)}h_1 \text{ contains } w_1^{(2)}; \text{ its derivative is } h_1) \\ &= (-0.269) \cdot 1 = -0.269, && \text{(insert } \delta^{(2)} = -0.269, h_1 = 1; \text{ multiply)} \\ \frac{\partial L}{\partial w_2^{(2)}} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial w_2^{(2)}} && \text{(the same full path, ending at } w_2^{(2)}) \\ &= \delta^{(2)} \cdot \frac{\partial z^{(2)}}{\partial w_2^{(2)}} && \text{(first two links = } \delta^{(2)}) \\ &= \delta^{(2)} \cdot h_2 && \text{(only } w_2^{(2)}h_2 \text{ contains } w_2^{(2)}) \\ &= (-0.269) \cdot 2 = -0.538, && \text{(insert } \delta^{(2)}, h_2 = 2; \text{ multiply)} \\ \frac{\partial L}{\partial b_2} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial b_2} && \text{(the same full path, ending at } b_2) \\ &= \delta^{(2)} \cdot \frac{\partial z^{(2)}}{\partial b_2} && \text{(first two links = } \delta^{(2)}) \\ &= \delta^{(2)} \cdot 1 && \text{(} b_2 \text{ is a plain added term: } \partial z^{(2)} / \partial b_2 = 1) \\ &= -0.269. && \text{(multiply)} \end{aligned}$$

The pattern that will repeat: *a weight's gradient is the error at its output times the activation at its input*; a bias's gradient is the error itself. Packed into matrix form, $\partial L / \partial W_2 = \delta^{(2)} h^\top = (-0.269, -0.538)$ and $\partial L / \partial b_2 = \delta^{(2)}$.

The signs make sense. All three gradients are negative, so the update rule (which steps *against* the gradient) will raise $w_1^{(2)}$, $w_2^{(2)}$, and b_2 ; since h_1 and h_2 are positive, every one of those changes raises $z^{(2)}$, which raises $\hat{y}$ — toward the truth 1. The calculus rediscovers common sense.

5.4 Back through the hidden layer

The hidden layer's parameters sit two stations further left, so the error must first be carried to h and then through the ReLU to $z^{(1)}$.

To h . The path is $h_j \rightarrow z^{(2)} \rightarrow \hat{y} \rightarrow L$, and in $z^{(2)} = w_1^{(2)}h_1 + w_2^{(2)}h_2 + b_2$ the coefficient of h_j is

the weight $w_j^{(2)}$:

$$\begin{aligned}
\frac{\partial L}{\partial h_1} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial h_1} && \text{(chain rule along } h_1 \rightarrow z^{(2)} \rightarrow \hat{y} \rightarrow L) \\
&= \delta^{(2)} \cdot w_1^{(2)} && \text{(first two links = } \delta^{(2)}; h_1 \text{'s coefficient is } w_1^{(2)}) \\
&= (-0.269) \cdot 2 = -0.538, && \text{(insert } \delta^{(2)} = -0.269, w_1^{(2)} = 2; \text{ multiply)} \\
\frac{\partial L}{\partial h_2} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial h_2} && \text{(the same path, ending at } h_2) \\
&= \delta^{(2)} \cdot w_2^{(2)} && \text{(} h_2 \text{'s coefficient is } w_2^{(2)}) \\
&= (-0.269) \cdot (-1) = 0.269. && \text{(a negative weight flips the sign)}
\end{aligned}$$

In packed form: $\partial L / \partial h = W_2^\top \delta^{(2)}$. The error travels backward along the same weights the signal traveled forward on — this is the propagation the name refers to. The signs again read sensibly: h_2 's output weight is -1 , so h_2 argues for no; the truth is yes; its positive gradient says h_2 should shrink.

Through the ReLU. From $z_i^{(1)}$ the path to L runs $z_i^{(1)} \rightarrow h_i \rightarrow z^{(2)} \rightarrow \hat{y} \rightarrow L$ — four links — and the newest link is $h_i = \text{ReLU}(z_i^{(1)})$, which holds entry by entry, with slope ReLU' (§3.3's derivative: 1 for positive scores, 0 for negative):

$$\begin{aligned}
\delta_i^{(1)} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial h_i} \cdot \frac{dh_i}{dz_i^{(1)}} && \text{(chain rule along } z_i^{(1)} \rightarrow h_i \rightarrow z^{(2)} \rightarrow \hat{y} \rightarrow L) \\
&= \frac{\partial L}{\partial h_i} \cdot \frac{dh_i}{dz_i^{(1)}} && \text{(the first three links are the } \partial L / \partial h_i \text{ just computed)} \\
&= \frac{\partial L}{\partial h_i} \cdot \text{ReLU}'(z_i^{(1)}), && (h_i = \text{ReLU}(z_i^{(1)}): \text{ this link's slope is } \text{ReLU}')$$

$$\begin{aligned}
\delta^{(1)} &= \begin{pmatrix} (-0.538) \cdot \text{ReLU}'(1) \\ 0.269 \cdot \text{ReLU}'(2) \end{pmatrix} && \text{(insert } \partial L / \partial h = (-0.538, 0.269)^\top, z^{(1)} = (1, 2)^\top) \\
&= \begin{pmatrix} -0.538 \cdot 1 \\ 0.269 \cdot 1 \end{pmatrix} = \begin{pmatrix} -0.538 \\ 0.269 \end{pmatrix}. && \text{(both scores positive: } \text{ReLU}' = 1 \text{ twice; multiply)}
\end{aligned}$$

Here §3.3's warning becomes a formula: on the input $(1, 3)^\top$, hidden neuron 1's score was -1.5 , so its factor would be $\text{ReLU}'(-1.5) = 0$ and it would receive *no* gradient — a silenced neuron does not learn.

The hidden parameters. Layer 1 has the same structure as layer 2 — $z_i^{(1)} = w_{i1}^{(1)}x_1 + w_{i2}^{(1)}x_2 + (b_1)_i$ — so the same pattern applies: error at the output times activation at the input, which for layer 1 is the input x itself. One entry in full — and note how long the full path from $w_{11}^{(1)}$ to L has become:

$$\begin{aligned}
\frac{\partial L}{\partial w_{11}^{(1)}} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial h_1} \cdot \frac{dh_1}{dz_1^{(1)}} \cdot \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}} && \text{(chain rule: five links now)} \\
&= \delta_1^{(1)} \cdot \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}} && \text{(the first four links are exactly } \delta_1^{(1)}) \\
&= \delta_1^{(1)} \cdot x_1 && \text{(only } w_{11}^{(1)}x_1 \text{ contains } w_{11}^{(1)}; \text{ its derivative is } x_1) \\
&= (-0.538) \cdot 2 = -1.076, && \text{(insert } \delta_1^{(1)} = -0.538, x_1 = 2; \text{ multiply)}
\end{aligned}$$

and all of them packed:

$$\begin{aligned}\frac{\partial L}{\partial W_1} &= \delta^{(1)} x^\top = \begin{pmatrix} -0.538 \\ 0.269 \end{pmatrix} \begin{pmatrix} 2 & 1 \end{pmatrix} && \text{(the repeating pattern, in matrix form)} \\ &= \begin{pmatrix} -0.538 \cdot 2 & -0.538 \cdot 1 \\ 0.269 \cdot 2 & 0.269 \cdot 1 \end{pmatrix} = \begin{pmatrix} -1.076 & -0.538 \\ 0.538 & 0.269 \end{pmatrix}, && \text{(entry } (i, j): \delta_i^{(1)} x_j) \\ \frac{\partial L}{\partial b_1} &= \delta^{(1)} = \begin{pmatrix} -0.538 \\ 0.269 \end{pmatrix}. && \text{(biases: the error itself)}\end{aligned}$$

5.5 All nine, and the general recipe

The walk is over, and every parameter has its gradient:

Parameter	Current value	$\partial L / \partial \theta$
$w_{11}^{(1)}$	0.5	-1.076
$w_{12}^{(1)}$	-1	-0.538
$w_{21}^{(1)}$	1	0.538
$w_{22}^{(1)}$	1	0.269
$(b_1)_1$	1	-0.538
$(b_1)_2$	-1	0.269
$w_1^{(2)}$	2	-0.269
$w_2^{(2)}$	-1	-0.538
b_2	1	-0.269

One of them can be checked without any calculus, because a gradient is a nudge ratio. Nudge $w_1^{(2)}$ from 2 to 2.01 and rerun the forward pass: $z^{(2)} = 2.01 \cdot 1 + (-1) \cdot 2 + 1 = 1.01$, $\hat{y} = \sigma(1.01) = 0.733$, $L = -\ln 0.733 = 0.3106$. The ratio is

$$\frac{0.3106 - 0.3133}{0.01} = -0.27 \approx -0.269,$$

matching the table (the small mismatch is the finite size of the nudge). The backward walk and the brute-force nudge agree.

The whole computation was three moves, repeated per layer. Writing them in general form takes one more symbol: $\odot$, the elementwise (Hadamard) product of *Essential Math for AI* — two vectors of the same shape multiply entry by entry, nothing more. The $\delta^{(1)}$ computation above already was one:

$$\begin{pmatrix} -0.538 \\ 0.269 \end{pmatrix} \odot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} -0.538 \cdot 1 \\ 0.269 \cdot 1 \end{pmatrix} = \begin{pmatrix} -0.538 \\ 0.269 \end{pmatrix}$$

— the left factor is $\partial L / \partial h$, the right factor holds the two ReLU' values. With that, and with $h^{(0)} = x$, the three moves in general form:

$$\delta^{(D)} = \frac{\partial L}{\partial \hat{y}} \odot \psi'(z^{(D)}) \quad \text{(the output error; for sigmoid + cross-entropy: } \hat{y} - y)$$

$$\delta^{(\ell)} = (W_{\ell+1}^\top \delta^{(\ell+1)}) \odot \varphi'(z^{(\ell)}) \quad \text{(back through the weights, then the activation)}$$

$$\frac{\partial L}{\partial W_\ell} = \delta^{(\ell)} (h^{(\ell-1)})^\top, \quad \frac{\partial L}{\partial b_\ell} = \delta^{(\ell)}. \quad \text{(gradients: local error } \times \text{ local input)}$$

(The first line as written needs an elementwise output activation; the softmax–cross-entropy pair collapses to the same $\hat{y} - y$.) On a batch, each example produces its own gradients and the batch gradient is their average — matching the batch loss of *What Learning Means*.

Now the accounting that makes deep learning possible. The backward walk visited each layer once, doing work comparable to the forward pass itself — one sweep, all nine gradients. The alternative would be the nudge check done nine times: nudge a parameter, rerun the network, compare — nine extra forward passes for nine parameters, and a million extra forward passes for a million. Backpropagation delivers every gradient for the price of roughly *one* extra pass, at any size. That is why networks with billions of parameters can be trained at all.

5.6 One step downhill, as a check

The gradients exist to be used. Apply *What Learning Means*’s update rule, $\theta_{\text{new}} = \theta_{\text{old}} - \eta \partial L / \partial \theta$, with learning rate $\eta = 0.1$, to all nine parameters at once. Every number that enters has a source: the current values are the running network as §4.2 defined it (W_1 , b_1 , W_2 , b_2), and the gradients are the ones just computed, tabulated in §5.5. One entry in full:

$$\begin{aligned}
 w_{11,\text{new}}^{(1)} &= w_{11,\text{old}}^{(1)} - \eta \cdot \frac{\partial L}{\partial w_{11}^{(1)}} && \text{(the update rule)} \\
 &= w_{11,\text{old}}^{(1)} - \eta \cdot \delta_1^{(1)} x_1 && (\text{\S 5.3's rule, applied in \S 5.4}) \\
 &= 0.5 - 0.1 \cdot ((-0.538) \cdot 2) && (\text{insert } w_{11}^{(1)} = 0.5, \eta = 0.1, \delta_1^{(1)} = -0.538, x_1 = 2) \\
 &= 0.5 - 0.1 \cdot (-1.076) && (\text{multiply } \delta_1^{(1)} x_1 \text{ — the table's gradient}) \\
 &= 0.5 - (-0.1076) && (\text{multiply by } \eta: 0.1 \cdot (-1.076) = -0.1076) \\
 &= 0.5 + 0.1076 = 0.6076. && (\text{minus a negative is plus; add})
 \end{aligned}$$

The other eight follow the same rule, packed per layer. Each chain first recalls the δ -form of its gradient — the rule was derived in §5.3 and applied to the hidden layer in §5.4 — and then inserts the numbers tabulated in §5.5:

$$\begin{aligned}
 W_{1,\text{new}} &= W_1 - \eta \frac{\partial L}{\partial W_1} && \text{(same rule, whole matrix)} \\
 &= W_1 - \eta \delta^{(1)} x^\top && (\text{\S 5.3's rule, applied in \S 5.4}) \\
 &= \begin{pmatrix} 0.5 & -1 \\ 1 & 1 \end{pmatrix} - 0.1 \begin{pmatrix} -1.076 & -0.538 \\ 0.538 & 0.269 \end{pmatrix} && (\text{numbers from \S 5.5}) \\
 &= \begin{pmatrix} 0.5 & -1 \\ 1 & 1 \end{pmatrix} - \begin{pmatrix} -0.1076 & -0.0538 \\ 0.0538 & 0.0269 \end{pmatrix} && (\text{multiply the gradient by } \eta = 0.1) \\
 &= \begin{pmatrix} 0.5 + 0.1076 & -1 + 0.0538 \\ 1 - 0.0538 & 1 - 0.0269 \end{pmatrix} && (\text{subtract: minus a negative is plus}) \\
 &= \begin{pmatrix} 0.6076 & -0.9462 \\ 0.9462 & 0.9731 \end{pmatrix}. && (\text{add})
 \end{aligned}$$

$$\begin{aligned}
b_{1,\text{new}} &= b_1 - \eta \frac{\partial L}{\partial b_1} && \text{(the rule for the biases)} \\
&= b_1 - \eta \delta^{(1)} && \text{(\S 5.3's bias rule, applied in \S 5.4)} \\
&= \begin{pmatrix} 1 \\ -1 \end{pmatrix} - 0.1 \begin{pmatrix} -0.538 \\ 0.269 \end{pmatrix} && \text{(numbers from \S 5.5's table)} \\
&= \begin{pmatrix} 1 \\ -1 \end{pmatrix} - \begin{pmatrix} -0.0538 \\ 0.0269 \end{pmatrix} && \text{(multiply by } \eta \text{)} \\
&= \begin{pmatrix} 1 + 0.0538 \\ -1 - 0.0269 \end{pmatrix} = \begin{pmatrix} 1.0538 \\ -1.0269 \end{pmatrix}. && \text{(subtract entry by entry; add)}
\end{aligned}$$

$$\begin{aligned}
W_{2,\text{new}} &= W_2 - \eta \frac{\partial L}{\partial W_2} && \text{(the rule for } W_2 \text{)} \\
&= W_2 - \eta \delta^{(2)} h^\top && (\partial L / \partial W_2 = \delta^{(2)} h^\top: \text{ derived in \S 5.3)} \\
&= \begin{pmatrix} 2 & -1 \end{pmatrix} - 0.1 \begin{pmatrix} -0.269 & -0.538 \end{pmatrix} && \text{(numbers from \S 5.5's table)} \\
&= \begin{pmatrix} 2 & -1 \end{pmatrix} - \begin{pmatrix} -0.0269 & -0.0538 \end{pmatrix} && \text{(multiply by } \eta \text{)} \\
&= \begin{pmatrix} 2 + 0.0269 & -1 + 0.0538 \end{pmatrix} = \begin{pmatrix} 2.0269 & -0.9462 \end{pmatrix}. && \text{(subtract; add)}
\end{aligned}$$

$$\begin{aligned}
b_{2,\text{new}} &= b_2 - \eta \frac{\partial L}{\partial b_2} = b_2 - \eta \delta^{(2)} && \text{(the rule; } \delta^{(2)} \text{ form from \S 5.3)} \\
&= 1 - 0.1 \cdot (-0.269) && \text{(insert } b_2 = 1, \eta, \delta^{(2)} = -0.269 \text{)} \\
&= 1 - (-0.0269) && \text{(multiply by } \eta \text{)} \\
&= 1 + 0.0269 = 1.0269. && \text{(minus a negative is plus; add)}
\end{aligned}$$

Rerun the forward pass on the same input as before, $x = (2, 1)^\top$ of §4.2, stage by stage:

$$\begin{aligned}
z^{(1)} &= W_{1,\text{new}} x + b_{1,\text{new}} && \text{(hidden pre-activations, new parameters)} \\
&= \begin{pmatrix} 0.6076 & -0.9462 \\ 0.9462 & 0.9731 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 1.0538 \\ -1.0269 \end{pmatrix} && \text{(insert the new } W_1, b_1 \text{ and the input)} \\
&= \begin{pmatrix} 0.6076 \cdot 2 - 0.9462 \cdot 1 \\ 0.9462 \cdot 2 + 0.9731 \cdot 1 \end{pmatrix} + \begin{pmatrix} 1.0538 \\ -1.0269 \end{pmatrix} && \text{(each row of } W_1 \text{ dots } x \text{)} \\
&= \begin{pmatrix} 0.2690 \\ 2.8655 \end{pmatrix} + \begin{pmatrix} 1.0538 \\ -1.0269 \end{pmatrix} = \begin{pmatrix} 1.3228 \\ 1.8386 \end{pmatrix}, && \text{(multiply; then add the biases)} \\
h &= \text{ReLU}(z^{(1)}) = \begin{pmatrix} 1.3228 \\ 1.8386 \end{pmatrix}, && \text{(both positive: ReLU passes both)} \\
z^{(2)} &= W_{2,\text{new}} h + b_{2,\text{new}} && \text{(the output pre-activation)} \\
&= 2.0269 \cdot 1.3228 - 0.9462 \cdot 1.8386 + 1.0269 && \text{(insert the new } W_2, b_2 \text{ and the fresh } h \text{)} \\
&= 2.6812 - 1.7397 + 1.0269 = 1.9684, && \text{(work out each product; add)} \\
\hat{y} &= \sigma(1.9684) \approx 0.877, && \text{(apply the output sigmoid)} \\
L &= -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})] && \text{(the cross-entropy loss of \S 4.3)} \\
&= -[1 \cdot \ln 0.877 + (1 - 1) \cdot \ln(1 - 0.877)] && \text{(insert } y = 1, \hat{y} = 0.877 \text{)} \\
&= -\ln 0.877 \approx 0.131. && \text{(the second term vanishes: } 1 - 1 = 0; \text{ evaluate)}
\end{aligned}$$

One step, and the loss fell from 0.313 to 0.131; the prediction rose from 0.731 to 0.877, toward the truth 1. Every parameter moved — including the hidden ones the loss formula never mentions. The full learning loop has now run by hand exactly once: forward pass (§4), backward pass (this section), update step (*What Learning Means*). Training a neural network is this loop, repeated.

5.7 Symbols

New persistent symbol: $\delta^{(\ell)}$ — see the [list of symbols](#). Local to this section:

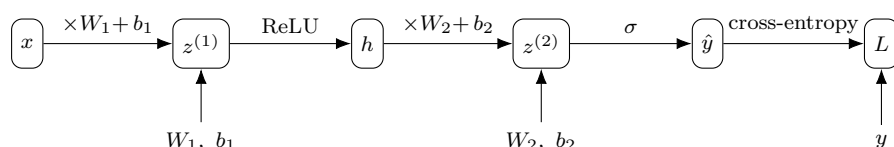
Local symbol	Meaning (this section)
$\partial L / \partial \theta$	how L responds to a small nudge of θ alone
$\delta^{(2)}, \delta^{(1)}$	the two errors: $-0.269; (-0.538, 0.269)^\top$
$w_{ij}^{(\ell)}, (b_\ell)_i$	entry (i, j) of W_ℓ ; entry i of b_ℓ
$\odot$	elementwise (Hadamard) product
η	the learning rate of <i>What Learning Means</i> ; 0.1 here
D	the number of layers, as in §4.4

6 The computational graph

§5 walked the chain rule backward by hand, reading the route off a diagram of stations. This section gives that diagram its proper name — the *computational graph* — and shows what the name buys: once a computation is recorded as a graph, the backward walk becomes a mechanical procedure that software can run on *any* architecture, with no formulas derived by hand. That machinery, *autograd*, is what every modern framework uses to train every modern network — and what it executes is exactly §5.

6.1 From computation to graph

The *computational graph* of a computation is the record of every elementary operation the computation performed: one *node* per value produced, and an *edge* from each input to each value computed from it. Recording the forward pass of §4.2 gives:



the running network's loss as a computational graph, at the station granularity of §5.1: every arrow points from an input to the value computed from it, and the label names the operation

This is the same picture as §5.1's road map — that road map *was* the computational graph, drawn coarse. In software the nodes are finer: $z^{(1)} = W_1x + b_1$ splits into a multiplication node and an addition node, because the graph's operations are exactly the ones the library knows a derivative for. The drawing above groups them per station because that is the granularity §5 walked.

Both passes of training are walks over this graph. The forward pass visits the nodes in the order the arrows point, computing each node as soon as its inputs are ready and storing its value. The backward pass plants $\partial L / \partial L = 1$ at the last node (L responds one-for-one to itself) and walks in reverse, multiplying along the way by each operation's *local derivative* — the one-step derivative the operation itself supplies. When it is done, every node holds two numbers of the same shape: its value and its gradient. §5 executed exactly this backward walk by hand.

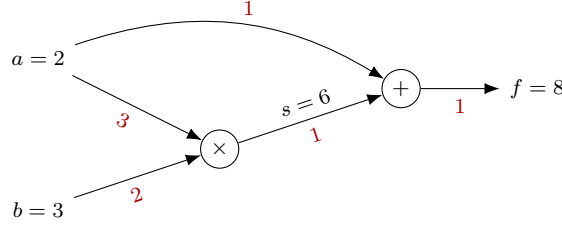
6.2 When paths branch, gradients add

The running network is a chain: from any parameter there is exactly one road to L . Real graphs branch — the same value can feed two operations. The chain rule for that case was built in *Essential Math for AI* (the multi-path chain rule): differentiate along each path separately, then *add* the paths.

The smallest branching graph, computed by hand. Take

$$f = a \cdot b + a, \quad a = 2, \quad b = 3,$$

and name the intermediate $s = a \cdot b$, so that $f = s + a$. Then a feeds *two* operations — the product and the sum:



black: the forward values; red: the gradient flowing backward along each edge. Two red numbers arrive at a — one per path — and add: $3 + 1 = 4$

Forward: $s = a \cdot b = 2 \cdot 3 = 6$, then $f = s + a = 6 + 2 = 8$. The gradient of f with respect to a :

$$\begin{aligned} \frac{df}{da} &= \frac{\partial f}{\partial s} \cdot \frac{ds}{da} + \frac{\partial f}{\partial a} && \text{(multi-path chain rule: one term per path — through } s, \text{ and direct)} \\ &= 1 \cdot b + 1 && (f = s + a: \text{ both partials are } 1; s = ab: ds/da = b) \\ &= 3 + 1 = 4. && (\text{insert } b = 3; \text{ add the paths}) \end{aligned}$$

The nudge agrees: $a = 2.01$ gives $f = 2.01 \cdot 3 + 2.01 = 8.04$, and $(8.04 - 8)/0.01 = 4$ — exact this time, because f is linear in a .

On the graph, this addition needs no special handling. The backward walk sends a gradient down *every* incoming edge; a value that fed two operations simply receives two deliveries and adds them. The branching is handled by arrival. The vanishing-gradients section will meet an architecture trick — the residual connection — built entirely on this rule.

6.3 Autograd: the machinery in software

Autograd — short for reverse-mode *automatic differentiation* — is the library machinery that records the computational graph while the forward pass runs, then executes the backward walk over the record. The contract it offers is the reason deep learning is programmable at all: the programmer writes *only the forward computation*, in ordinary code; the library watches it run, builds the graph, and on the call `loss.backward()` (PyTorch’s name for it) walks the graph backward — after which every parameter holds its gradient in a field called `.grad`. §5, executed mechanically, for any architecture anyone writes.

Concretely, here is the record — the *tape*, as libraries call it — as it stands after the forward pass of §4.2 has run:

Step	The line the programmer wrote	Operation recorded	Value stored
1	<code>z1 = W1 @ x + b1</code>	multiply-add; reads W_1, x, b_1	$z^{(1)} = (1, 2)^\top$
2	<code>h = relu(z1)</code>	ReLU; reads $z^{(1)}$	$h = (1, 2)^\top$
3	<code>z2 = W2 @ h + b2</code>	multiply-add; reads h, W_2, b_2	$z^{(2)} = 1$
4	<code>yhat = sigmoid(z2)</code>	σ ; reads $z^{(2)}$	$\hat{y} = 0.731$
5	<code>loss = cross_entropy(yhat, y)</code>	cross-entropy; reads $\hat{y}, y$	$L = 0.313$

The call `loss.backward()` replays the tape in reverse, one step at a time, each step multiplying the incoming gradient by the local derivative of the operation it recorded:

- Step 5 (cross-entropy): the walk starts from $\partial L / \partial L = 1$; times the local slope $-y/\hat{y} + (1 - y)/(1 - \hat{y}) = -1.368$, delivered to $\hat{y}$. Nothing goes to y — a label is never updated.
- Step 4 (σ): multiply by $\sigma'(1) = 0.197$; delivers $(-1.368) \cdot 0.197 = -0.269 = \delta^{(2)}$ to $z^{(2)}$.

- Step 3 (multiply-add): delivers $\delta^{(2)}h^\top = (-0.269, -0.538)$ to W_2 , $\delta^{(2)} = -0.269$ to b_2 , and $W_2^\top \delta^{(2)} = (-0.538, 0.269)^\top$ to h .
- Step 2 (ReLU): multiply by ReLU' of $(1, 2)^\top$, which is $(1, 1)^\top$; delivers $\delta^{(1)} = (-0.538, 0.269)^\top$ to $z^{(1)}$.
- Step 1 (multiply-add): delivers $\delta^{(1)}x^\top$ to W_1 and $\delta^{(1)}$ to b_1 ; the delivery to x is skipped.

When the replay ends, every parameter's `.grad` holds exactly §5.5's table — reproduced without a single derivation: each step used only its own recorded inputs and its own built-in local derivative.

That is the deeper point of the graph view: no step ever needs to know the whole network. σ ships $\sigma' = \sigma(1 - \sigma)$, ReLU ships its 0/1 rule, a product ships “the other factor” — and the graph's edges do all the bookkeeping. The chain rule stops being a formula to derive and becomes a routing rule: multiply along every edge, walking right to left.

Three ways to obtain derivatives have now appeared, and the comparison is the point:

- *By hand*, as §5 did: exact, but the derivation is labor, redone for every new architecture.
- *By nudging*, as §5.5's check did: no calculus needed, but approximate — and one forward rerun *per parameter*, which at a million parameters is a million reruns.
- *By autograd*: exact like the hand derivation, effortless like the nudge, and all parameters in one backward pass.

One cost is worth knowing. The backward walk needs the stored forward values (the middle column of the table above), so the library must keep every intermediate alive until `backward()` has run. Training therefore needs far more memory than merely running the network — and since a batch multiplies every intermediate by B (§4.5's batch matrices), the batch size of *What Learning Means* is in practice capped by exactly this memory.

6.4 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
node, edge	a value produced; “this input was used to compute that value”
s, f, a, b	the branching example: $s = ab = 6$, $f = s + a = 8$
local derivative	the one-operation derivative each operation ships with
$\partial L / \partial L = 1$	the seed the backward pass plants at L
autograd	reverse-mode automatic differentiation: record forward, walk backward
the tape	the recorded step list; <code>backward()</code> replays it in reverse

7 Weight initialization

The training loop — forward pass, backward pass, update — transforms the parameters it is handed; nothing in it says where the *first* parameters come from. *Weight initialization* is the choice of the parameter values training starts from. For the one-parameter model of *What Learning Means* almost any start worked: beginning farther away just cost a few more steps. A network is less forgiving, in two separate ways: a start with the wrong *pattern* freezes neurons into permanent clones, and a start with the wrong *scale* shrinks or inflates signals exponentially with depth. This section shows both failures in small numbers, then the standard fix — random values, sized to the layer’s fan-in.

7.1 Failure one: symmetry never breaks itself

The tempting start is the simplest one: set every weight to the same constant. Try it on the running network’s shape — two inputs, two hidden neurons, one output — with every weight 0.5 and every bias 0:

$$W_1 = \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad W_2 = (0.5 \quad 0.5), \quad b_2 = 0,$$

and send §4.2’s input $x = (2, 1)^\top$, label $y = 1$, through the forward pass:

$$\begin{aligned} z^{(1)} &= W_1 x + b_1 && \text{(the hidden layer, as in §4.1)} \\ &= \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} && \text{(insert the all-equal start and the input)} \\ &= \begin{pmatrix} 0.5 \cdot 2 + 0.5 \cdot 1 \\ 0.5 \cdot 2 + 0.5 \cdot 1 \end{pmatrix} = \begin{pmatrix} 1.5 \\ 1.5 \end{pmatrix}, && \text{(identical rows compute identical sums)} \\ h &= \text{ReLU}(z^{(1)}) = \begin{pmatrix} 1.5 \\ 1.5 \end{pmatrix}, && \text{(still identical)} \\ z^{(2)} &= 0.5 \cdot 1.5 + 0.5 \cdot 1.5 + 0 = 1.5, && \text{(the output layer as usual)} \\ \hat{y} &= \sigma(1.5) \approx 0.818, && \text{(the output sigmoid)} \\ L &= -\ln 0.818 \approx 0.201. && (y = 1: §4.3’s second term vanishes) \end{aligned}$$

The two hidden neurons agree, and they must: with identical incoming weights and biases they are the same function of every input. The hope would be that training pulls them apart. It cannot. Walk the backward pass once, with §5.5’s recipe:

$$\begin{aligned} \delta^{(2)} &= \hat{y} - y && (\S 5.2\text{’s collapse}) \\ &= 0.818 - 1 = -0.182, && \text{(insert } \hat{y}, y) \\ \frac{\partial L}{\partial h} &= W_2^\top \delta^{(2)} = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} \cdot (-0.182) = \begin{pmatrix} -0.091 \\ -0.091 \end{pmatrix}, && (\S 5.4\text{’s rule; equal outgoing weights)} \\ \delta^{(1)} &= \frac{\partial L}{\partial h} \odot \text{ReLU}'(z^{(1)}) = \begin{pmatrix} -0.091 \\ -0.091 \end{pmatrix}, && \text{(both scores } 1.5 > 0: \text{ factors } 1) \\ \frac{\partial L}{\partial W_1} &= \delta^{(1)} x^\top = \begin{pmatrix} -0.182 & -0.091 \\ -0.182 & -0.091 \end{pmatrix}, \quad \frac{\partial L}{\partial b_1} = \delta^{(1)}. && \text{(two identical rows)} \end{aligned}$$

The verdict is in the last line: the gradient’s two rows are identical, so the update rule subtracts the same amounts from rows that were already equal — and after the step they are equal again. The

same holds for W_2 's two entries: $\partial L / \partial W_2 = \delta^{(2)} h^\top = (-0.273, -0.273)$. Nothing in the loop can ever tell neuron 1 and neuron 2 apart: neurons that start identical receive identical gradients and remain identical *forever*. A width-two hidden layer does the work of a width-one layer; at width 1000, of one neuron copied 1000 times. (All zeros is the extreme case: besides the cloning, $h = 0$ would kill $\partial L / \partial W_2 = \delta^{(2)} h^\top$ entirely.)

The cure is to start the weights *random*. Randomness here is not decoration — it is the symmetry breaker: two random rows already differ at step 0, so their gradients differ, and training can push them toward different detectors. Biases do not need it: once the weights differ, the neurons differ. Standard practice is weights random, biases zero.

7.2 Failure two: the wrong scale compounds with depth

Random settles the pattern; the *size* of the random numbers is the second trap. Every layer multiplies its input by W , so each layer multiplies the signal's size by some factor — and whatever that factor is, depth raises it to a power. Watch it in the simplest possible chain: D layers, each one neuron wide, the same weight w in every layer, no bias, identity activation (a pure size experiment — §3.1 says such a chain collapses, which is exactly what makes its output easy to read):

$$h^{(\ell)} = w h^{(\ell-1)}, \quad h^{(D)} = w^D x.$$

Start with $x = 1$:

w	w^{10}	w^{30}
0.5	0.00098	$9.3 \cdot 10^{-10}$
0.9	0.35	0.042
1.0	1	1
1.1	2.59	17.4
2.0	1024	$1.1 \cdot 10^9$

A per-layer factor only slightly below 1 starves the deep layers of signal; slightly above 1, and the numbers grow toward overflow. Depth turns a mild factor into an exponential. And the same multiplication hits the *gradients* on their way back — §5.5's recipe multiplies by $W_{\ell+1}^\top$ at every layer — so a bad scale attacks learning from both ends. (That story continues in the next section.)

With real layers the per-layer factor is set by two things. One is the *fan-in* n_{in} : the number of inputs a neuron reads — equally, the number of weights it owns, since each input gets one weight. It is fixed by the architecture: the running network's hidden neurons have fan-in 2; a layer reading 10,000 activations has fan-in 10,000. The other is the weights' *typical size* s : the weights are drawn at random, so no single weight has a fixed value, but the draw has a scale — s is the standard deviation of the distribution the weights are drawn from (*Essential Math for AI's* σ), the size a typical draw has with the sign ignored. Drawing with $s = 0.01$ scatters the weights around ± 0.01 . Now put the two together. A neuron with n_{in} inputs computes $z = \sum_j w_j x_j$ — a sum of n_{in} terms with random signs. Such a sum does not grow like n_{in} : the cancellations keep its typical size near $\sqrt{n_{\text{in}}}$ times one term's size, because variances of independent terms add and size is the square root of variance (*Essential Math for AI*). The per-layer factor is therefore roughly $s\sqrt{n_{\text{in}}}$ — and that is the quantity that must sit near 1.

7.3 The fix: random, sized to the fan-in

Set $s\sqrt{n_{\text{in}}} \approx 1$ and the recipe writes itself:

$$s \approx \frac{1}{\sqrt{n_{\text{in}}}}$$

— draw each weight at random, with typical size $1/\sqrt{n_{\text{in}}}$, where n_{in} is the neuron’s fan-in. The two standard schemes are this rule with small refinements:

- *Xavier (Glorot) initialization* — variance $1/n_{\text{in}}$ (or $2/(n_{\text{in}} + n_{\text{out}})$, balancing both directions); the default for tanh and sigmoid layers.
- *He initialization* — variance $2/n_{\text{in}}$; the default for ReLU layers. The extra factor 2 compensates ReLU: zeroing the negative half removes half the signal’s variance, so the weights start with twice the variance to make up for it.

Which distribution supplies the randomness is a genuinely free choice, because the argument above used only two properties of the draw: mean 0 (no preferred sign) and the chosen variance. Two shapes are standard: the Gaussian $\mathcal{N}(0, s^2)$ — the bell curve of *Essential Math for AI*’s noise model — and the uniform distribution on $[-a, a]$, with $a = \sqrt{3}s$ so that its variance $a^2/3$ equals the same s^2 . Frameworks ship both under each scheme’s name (Xavier-uniform, Xavier-normal, and so on), and the choice between them makes no practical difference.

The numbers this produces are small. A layer reading $n_{\text{in}} = 10,000$ activations gets Xavier scale $1/\sqrt{10000} = 0.01$: weights around ± 0.01 , not around ± 1 . (Our running network’s hidden layer has fan-in 2, scale $1/\sqrt{2} \approx 0.7$ — §4.2’s hand-picked weights sit within an order of magnitude of that, one reason §5’s single step went so smoothly.) In practice the framework applies these formulas the moment a layer is created; nobody types them. But when a fresh network refuses to train — the loss flat, or turning into NaN (the floating-point overflow marker) within a few steps — a wrong initialization is among the first suspects.

7.4 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
n_{in} (fan-in)	the number of inputs a neuron reads
s	the typical size of a freshly drawn weight
w^D	the toy chain’s output factor: one weight w , D layers
symmetry	identical parameters $\rightarrow$ identical gradients $\rightarrow$ identical forever
Xavier / He	variance $1/n_{\text{in}}$; variance $2/n_{\text{in}}$ (for ReLU)

8 Vanishing and exploding gradients

§7.2 watched the *signal* shrink or blow up on its way forward. The same fate awaits the *gradient* on its way back — and this, not a lack of expressive power, is what kept deep networks untrainable for decades. *Vanishing gradients* is the failure mode in which the early layers' gradients shrink exponentially with depth, so the front of the network stops learning; *exploding gradients* is the same product running upward, making steps too large to survive. This section locates the product inside §5.5's recipe, computes how fast the sigmoid kills it, and collects the remedies — most of them already on the table.

8.1 The gradient is a product with D factors

§5.5's recursion moves the error down one layer at a time,

$$\delta^{(\ell)} = (W_{\ell+1}^\top \delta^{(\ell+1)}) \odot \varphi'(z^{(\ell)}),$$

and every application multiplies by two things: a weight factor $W^\top$ and a slope factor φ' . To see what that does over depth, take the simplest deep network: the chain of §7.2, one neuron per layer, weight $w^{(\ell)}$ in layer ℓ — but now with its activation φ back in, $h^{(\ell)} = \varphi(w^{(\ell)}h^{(\ell-1)})$. Unroll the recursion from the top:

$$\begin{aligned} \delta^{(D-1)} &= \delta^{(D)} \cdot w^{(D)} \cdot \varphi'(z^{(D-1)}) && \text{(the recipe: scalar } W^\top = \text{the weight)} \\ \delta^{(D-2)} &= \delta^{(D)} \cdot w^{(D)} w^{(D-1)} \cdot \varphi'(z^{(D-1)}) \varphi'(z^{(D-2)}) && \text{(apply it once more)} \\ &\vdots \\ \delta^{(1)} &= \delta^{(D)} \cdot \underbrace{w^{(D)} \dots w^{(2)}}_{D-1 \text{ weight factors}} \cdot \underbrace{\varphi'(z^{(D-1)}) \dots \varphi'(z^{(1)})}_{D-1 \text{ slope factors}}. && \text{(all the way down)} \end{aligned}$$

The front layer's error — and with it every front-layer parameter gradient, since $\partial L / \partial w^{(1)}$ carries $\delta^{(1)}$ as a factor — is the output error times a product of $2(D-1)$ numbers. With matrices the unrolling looks the same, one $W^\top$ and one φ' per layer, and the *sizes* multiply the same way (§7.2's per-layer factor, now acting on the gradient). Products of many factors were §7.2's lesson: whatever the per-factor size is, depth raises it to a power.

8.2 Vanishing: the sigmoid's quarter ceiling

Now put the sigmoid into the slope factors. §3.3's table says $\sigma' = \sigma(1-\sigma)$ peaks at 0.25, reached only at $z = 0$, and decays toward 0 in both tails. So in a network of sigmoid hidden layers, *every* slope factor is at most a quarter — at best. After k layers,

$$\begin{aligned} \varphi'(z^{(k)}) \dots \varphi'(z^{(1)}) &\leq 0.25 \cdot 0.25 \dots 0.25 && \text{(each of the } k \text{ factors is at most 0.25)} \\ &= 0.25^k. && (k \text{ copies multiplied}) \end{aligned}$$

k (sigmoid layers crossed)	ceiling 0.25^k
5	0.00098
10	$9.5 \cdot 10^{-7}$
20	$9.1 \cdot 10^{-13}$
30	$8.7 \cdot 10^{-19}$

And this is the *best* case, every pre-activation sitting exactly at 0; saturated neurons contribute nearly 0 instead of 0.25. Even our two-layer run paid one such factor: §5.2’s slope was $\sigma'(1) = 0.197$, already below the ceiling because $z^{(2)} = 1$ sits off center.

The consequence is a network at war with its own learning rate. At ten sigmoid layers the front gradients are about a million times smaller than the back ones, yet the update rule of *What Learning Means* serves every parameter with the same η . Set η for the back layers and the front ones move by nothing — frozen; set η big enough for the front and the back layers take steps a million times too large. Historically this is exactly what happened: deep sigmoid networks trained their last few layers and left the rest at their initialization, and depth stalled on this arithmetic until the remedies below arrived.

8.3 Exploding: the same product, upward

The weight factors have no ceiling. If their typical size sits above 1 — a bad initialization, or weights drifting large during training — §7.2’s table now reads on the gradient side: factor 1.1 over 30 layers multiplies the gradient by 17; factor 2, by 10^9 . A gradient 10^9 times too large makes the update rule take a step 10^9 times too long — *What Learning Means* showed what oversized steps do (overshoot, then divergence), and at these magnitudes the arithmetic simply overflows into NaN (§7.3). One asymmetry is worth noting: ReLU will cap the slope factors at 1, but it does nothing to the weight factors — exploding remains possible in a ReLU network, which is why a blunt guard (gradient clipping, below) exists at all.

8.4 The remedies, most already met

Every remedy attacks the product. Two are already in this chapter’s toolbox:

- *ReLU* (§3.3, §3.4): its slope is exactly 1 on the whole positive side, so the slope half of the product stops decaying entirely. This resolves the battle §3.3 previewed — and it is *the* reason ReLU is the hidden-layer default.
- *Fan-in initialization* (§7.3): starts the weight factors at typical size 1, so the product begins its life near 1 instead of already collapsed or already huge.

Those two protect the start; training can still drift. Three more tools, each with its own place ahead:

- *Normalization* (a section ahead): re-centers each layer’s pre-activations *during* training, holding the slope factors away from the saturated tails.
- *Gradient clipping* (a section ahead): caps the assembled gradient’s size just before the step — the guard on the exploding side.
- *Residual connections* (the architectures chapter ahead): add a shortcut edge that carries x straight past the layer, output = $x +$ (the layer’s work). By §6.2’s rule the gradient flows back through the shortcut too — one path with *no* factors on it at all. The product of depth becomes optional.

One disease — multiplicative depth — and every cure either tames the factors (ReLU, initialization, normalization), caps the result (clipping), or builds a road around the product (residual connections).

8.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$w^{(\ell)}$	the toy chain's weight in layer ℓ
weight factor, slope factor	the $W^\top$ and φ' each layer multiplies in
0.25^k	the ceiling on k sigmoid slope factors
vanishing / exploding	the product collapsing to 0 / growing without bound

9 Batches and epochs

What Learning Means chose *how many* examples feed a step — the batch, and its noise trade-off — and §4.5 showed how a batch physically runs: one matrix multiplication. This section connects the two into the working vocabulary of training: what one step does from start to finish, what an epoch is, why the data is shuffled, and what sets the batch size in practice. These are the units every training log is written in.

9.1 One step = one batch

A *step* (also called an *iteration*) is one full turn of the training loop on one batch of B examples: assemble the batch matrix X (§4.5), run the forward pass, score every row, run the backward pass, update once. The only new question is what loss the backward pass differentiates. It is the *average* of the per-example losses — the batch loss of *What Learning Means* — and averaging carries straight through the derivative:

$$\begin{aligned} L_{\text{batch}} &= \frac{1}{B} \sum_{i=1}^B L_i && \text{(the batch loss: the average score)} \\ \frac{\partial L_{\text{batch}}}{\partial \theta} &= \frac{\partial}{\partial \theta} \left[\frac{1}{B} \sum_{i=1}^B L_i \right] && \text{(differentiate both sides)} \\ &= \frac{1}{B} \sum_{i=1}^B \frac{\partial L_i}{\partial \theta}. && \text{(sum rule; the constant } 1/B \text{ passes through)} \end{aligned}$$

The batch gradient is the average of the per-example gradients: each example runs §5's backward pass on its own stored values, and the results are averaged. (In software the B backward passes run batched, exactly like §4.5's forward — the average comes out directly.)

Watch one step at $B = 2$, using the two examples already run. Example 1 is §4.2's $x = (2, 1)^\top$ with $y = 1$; its loss 0.313 and its nine gradients are §5.5's table. Example 2 is the other input, $x = (1, 3)^\top$ — and give it its label now: $y = 0$. §4.2 read its output $\hat{y} = 0.119$ as a firm no; make that the truth. Its loss:

$$\begin{aligned} L_2 &= -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})] && \text{ (§4.3's loss)} \\ &= -[0 \cdot \ln 0.119 + (1 - 0) \cdot \ln(1 - 0.119)] && \text{ (insert } y = 0, \hat{y} = 0.119) \\ &= -\ln 0.881 \approx 0.127. && \text{ (this time the } \textit{first} \text{ term vanishes)} \end{aligned}$$

Its backward pass, by §5.5's recipe on the stored values of §4.2's second run ($z^{(1)} = (-1.5, 3)^\top$, $h = (0, 3)^\top$):

$$\begin{aligned} \delta^{(2)} &= \hat{y} - y = 0.119 - 0 = 0.119, && \text{ (§5.2's collapse)} \\ \frac{\partial L_2}{\partial W_2} &= \delta^{(2)} h^\top = 0.119 \cdot (0, 3) = (0, 0.357), \quad \frac{\partial L_2}{\partial b_2} = 0.119, && \text{ (§5.3's rule)} \\ \frac{\partial L_2}{\partial h} &= W_2^\top \delta^{(2)} = \begin{pmatrix} 2 \\ -1 \end{pmatrix} \cdot 0.119 = \begin{pmatrix} 0.238 \\ -0.119 \end{pmatrix}, && \text{ (§5.4)} \\ \delta^{(1)} &= \frac{\partial L_2}{\partial h} \odot \text{ReLU}'((-1.5, 3)^\top) = \begin{pmatrix} 0.238 \cdot 0 \\ -0.119 \cdot 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -0.119 \end{pmatrix}, \quad (z_1^{(1)} = -1.5 < 0: \text{ factor } 0) \\ \frac{\partial L_2}{\partial W_1} &= \delta^{(1)} x^\top = \begin{pmatrix} 0 & 0 \\ -0.119 & -0.357 \end{pmatrix}, \quad \frac{\partial L_2}{\partial b_1} = \begin{pmatrix} 0 \\ -0.119 \end{pmatrix}. && \text{ (a whole row of zeros)} \end{aligned}$$

Now average, parameter by parameter:

Parameter	$(2, 1)^\top, y = 1$	$(1, 3)^\top, y = 0$	batch average
$w_{11}^{(1)}$	-1.076	0	-0.538
$w_{12}^{(1)}$	-0.538	0	-0.269
$w_{21}^{(1)}$	0.538	-0.119	0.210
$w_{22}^{(1)}$	0.269	-0.357	-0.044
$(b_1)_1$	-0.538	0	-0.269
$(b_1)_2$	0.269	-0.119	0.075
$w_1^{(2)}$	-0.269	0	-0.135
$w_2^{(2)}$	-0.538	0.357	-0.091
b_2	-0.269	0.119	-0.075

The batch loss is $L_{\text{batch}} = (0.313 + 0.127)/2 = 0.220$, and the update rule steps on the right-hand column. Two things in the table are worth staring at. First, on $w_{22}^{(1)}$, $w_2^{(2)}$, b_2 and others, the two examples pull in *opposite* directions — this is the batch-gradient disagreement of *What Learning Means*, now with its machinery visible; the average is the compromise the step actually takes. Second, example 2 contributes exactly *nothing* to hidden neuron 1 — the row of zeros is §5.4’s silenced-neuron warning happening for real. Inside a batch that is harmless: example 1 keeps neuron 1 learning. A neuron is dead only if its score is negative for *every* example.

9.2 The epoch, and the shuffle

An *epoch* is one complete pass through the training set: with N examples and batches of B , that is N/B steps. The two units live on different scales — a dataset of $N = 50,000$ with $B = 100$ makes an epoch 500 steps, and a training run lasts tens of epochs, so hundreds of thousands of updates.

Between epochs the data is *shuffled*: the examples are put in a fresh random order, so the batches of epoch 2 are not the batches of epoch 1. The reason is the noise story of *What Learning Means*: each batch’s gradient is a noisy estimate, and if the same batches repeat in the same order, the same noise repeats with them — the walk can settle into a cycle instead of averaging its noise away. Fresh batches, fresh noise, cleaner average. (If B does not divide N , the last batch of an epoch is simply smaller, or dropped; a bookkeeping detail.)

9.3 What sets B in practice

Three forces, two of them met in this chapter:

- *Noise* (*What Learning Means*): smaller B means a noisier gradient; the noise shrinks like $1/\sqrt{B}$ as the batch grows.
- *Throughput* (§4.5): the batch runs as one matrix multiplication, and the hardware digests one big multiplication far faster than B small ones — larger batches keep the machine busy.
- *Memory* (the tape of §6): every stored intermediate is B rows tall and must live until `backward()`; the batch size is capped by what fits.

In practice B is often simply the largest value the memory allows, rounded to a convenient size — a few dozen to a few hundred is typical for a single machine, and the noise floor is then managed with the learning rate rather than with B itself.

9.4 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
B, N	the batch size; the number of training examples
step (iteration)	one batch through the loop: forward, backward, one update
epoch	one complete pass through all N examples: N/B steps
L_{batch}, L_i	the average loss of the batch; example i 's own loss
shuffle	re-randomize the example order between epochs

10 Optimizers

Every update so far has been the plain rule of *What Learning Means*: $\theta_{\text{new}} = \theta_{\text{old}} - \eta g$, where g is shorthand for the gradient $\partial L / \partial \theta$ that backpropagation delivers. One convention up front: a fresh g is computed at every step — each batch delivers a new one (§9.1) — and when the step number matters we index it, g_t being the gradient computed at step t . The plain rule is *memoryless*: written with indices it is $\theta_{t+1} = \theta_t - \eta g_t$, using the current gradient alone and forgetting it. An *optimizer* is the rule that turns gradients into parameter updates — and the plain rule is only the simplest member of the family. The rest of the loop never changes; the last stage is a swappable part. This section builds the two upgrades that dominate practice: *momentum*, which remembers direction, and *Adam*, which gives every parameter its own step size — and both differ from the plain rule in exactly one respect: they keep a memory of the earlier g 's. (All these rules act on each parameter independently; for vectors and matrices, read every operation entrywise.)

10.1 Two complaints about the plain rule

Both problems are already on record.

The steps jitter. Batch gradients are noisy estimates that disagree step to step, and §9.1's table holds a concrete case: on the parameter $w_{22}^{(1)}$, example $(2, 1)^\top$'s gradient is $+0.269$ while example $(1, 3)^\top$'s is -0.357 . Train with $B = 1$, the two examples taking turns, and that parameter is shoved in the opposite direction on every step ($\eta = 0.1$):

$$g_1 = 0.269, \quad g_2 = -0.357, \quad g_3 = 0.269, \quad \dots \quad \text{steps} - \eta g_t: \quad -0.027, +0.036, -0.027, \dots$$

(Odd steps are example $(2, 1)^\top$'s turn, even steps example $(1, 3)^\top$'s. Strictly, the parameters move between steps, so example $(2, 1)^\top$'s gradient at step 3 would no longer be exactly 0.269; the numbers are frozen here because the disease on display is the sign flip, not the third decimal.) The zigzag moves about 0.03 per step, yet the two examples' *combined* preference is only the average gradient -0.044 — a true step of $+0.0044$. Most of each step is spent undoing the previous one. *What Learning Means* watched the same disease from the outside, as a walk bouncing between two values instead of settling. The plain rule follows each noisy gradient in full; the noise goes straight into the path.

One η serves parameters of wildly different scale. §8.2 ended in a war: front-layer gradients a million times smaller than back-layer ones, and a single learning rate that cannot fit both. The plain rule multiplies every gradient by the same η , so the mismatch passes straight into the steps.

Momentum attacks the first complaint; Adam attacks both.

10.2 Momentum: remember the direction

Momentum is the optimizer that keeps a *velocity* v — a running blend of past gradients — and steps along the velocity instead of the raw gradient:

$$v_t = \beta v_{t-1} + g_t, \quad \theta_{\text{new}} = \theta_{\text{old}} - \eta v_t, \quad v_0 = 0,$$

where $\beta \in [0, 1)$ (typically 0.9) is the *momentum coefficient*: how much of the old direction survives into each new step. At $\beta = 0$ the rule is plain gradient descent again. The name is the physics picture: a heavy ball keeps moving the way it was already moving, and small bumps average out.

What the blending does is easiest to see on two extreme gradient sequences — pure noise (g_t alternating $+1, -1, +1, \dots$) and pure signal ($g_t = 1$ every step). The alternating case is §10.1’s zigzag, idealized. Run $v_t = 0.9 v_{t-1} + g_t$ by hand:

t	v_t when g_t alternates $+1, -1, \dots$	v_t when $g_t = 1$ throughout
1	1	1
2	-0.1	1.9
3	0.91	2.71
4	-0.181	3.44
5	0.837	4.10
6	-0.247	4.69
$\vdots$	settles at ± 0.53	climbs toward 10

The alternating column shrinks: sign-flipping noise partly cancels *inside* the velocity, and the jitter settles at about half the raw amplitude. The constant column grows: a direction that persists accumulates, and its limit is exact:

$$\begin{aligned}
 v_\infty &= g + \beta g + \beta^2 g + \dots && \text{(each old gradient decays by } \beta \text{ per step)} \\
 \beta v_\infty &= \beta g + \beta^2 g + \dots && \text{(multiply the whole line by } \beta) \\
 v_\infty - \beta v_\infty &= g && \text{(subtract: every term cancels except the first)} \\
 v_\infty &= \frac{g}{1 - \beta} = \frac{1}{1 - 0.9} = 10. && \text{(solve; insert } g = 1, \beta = 0.9)
 \end{aligned}$$

So momentum treats the two components of a noisy gradient differently: the part that flips sign is damped, the part that persists is amplified up to tenfold. The walk stops jittering and leans into its long-term direction — exactly the medicine for the bounce of *What Learning Means*.

10.3 Adam: a step size per parameter

Adam (*adaptive moment estimation*) is the optimizer that keeps *two* running blends per parameter — the average gradient, and the average *squared* gradient — and divides one by the square root of the other:

$$\begin{aligned}
 m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, & s_t &= \beta_2 s_{t-1} + (1 - \beta_2) g_t^2, \\
 \theta_{\text{new}} &= \theta_{\text{old}} - \eta \cdot \frac{m_t}{\sqrt{s_t} + \varepsilon},
 \end{aligned}$$

with $\beta_1 \approx 0.9$ (momentum’s memory, now averaged), $\beta_2 \approx 0.999$ (a longer memory for the size), and $\varepsilon \approx 10^{-8}$ a tiny guard against dividing by zero. (m and s are the “moments” of the name: the running estimates of the gradient’s mean and of its mean square.)

The heart of the rule is the division. $\sqrt{s_t}$ is the typical *size* of this parameter’s own recent gradients, so $m_t / \sqrt{s_t}$ is the gradient measured *in units of its own typical size* — a number of order ± 1 for every parameter, whatever its absolute scale. Watch the steady state: if a parameter’s gradient holds a constant value g , the blends settle at $m \approx g$ and $s \approx g^2$, and

$$\begin{aligned}
 \text{step} &= \eta \cdot \frac{m}{\sqrt{s} + \varepsilon} && \text{(the rule)} \\
 &\approx \eta \cdot \frac{g}{\sqrt{g^2} + \varepsilon} && \text{(insert the steady-state } m \approx g, s \approx g^2) \\
 &= \eta \cdot \frac{g}{|g| + \varepsilon} \approx \pm \eta. && (\sqrt{g^2} = |g|; g/|g| \text{ is the sign})
 \end{aligned}$$

Every parameter moves by about η per step, in its own downhill direction, regardless of how large or small its gradient is. Two parameters with steady gradients 1 and 0.001: under the plain rule with $\eta = 0.001$ their steps are 0.001 and 10^{-6} — a thousandfold gap; under Adam, both step about 0.001. §8.2’s scale war is neutralized — not the loss of information (a gradient of exactly 0 still moves nothing), but the mismatch of scales. One refinement completes the rule: because m and s start at 0, their early values undershoot, and Adam divides each by $(1 - \beta^t)$ to compensate — a warm-up correction the framework applies silently.

10.4 Choosing in practice

Optimizer	Keeps per parameter	Update	Fixes
plain SGD	nothing	$-\eta g$	(the baseline)
momentum	v	$-\eta v$	jitter from noisy gradients
Adam	m, s	$-\eta m/(\sqrt{s} + \varepsilon)$	jitter and scale mismatch

Adam is the default choice, especially for large networks whose layers have very different gradient scales; its common variant *AdamW*, which couples it with the weight decay of *What Learning Means*, is what essentially all large language models train under. Plain SGD with momentum remains competitive in some settings (much of computer vision) when carefully tuned. One thing no optimizer removes: the learning rate. η is still there in every rule above, and how it is varied *during* training is the next section.

10.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
g, g_t	shorthand for the gradient $\partial L/\partial \theta$ (at step t)
v_t, β	momentum’s velocity; how much of it survives a step (0.9)
m_t, s_t	Adam’s running average gradient and average squared gradient
β_1, β_2	their memory lengths (0.9; 0.999)
ε	the tiny divide-by-zero guard ($\approx 10^{-8}$)

11 Learning-rate schedules and warmup

Every rule of §10 still contains η , and so far η has been one constant number. A *learning-rate schedule* is a plan, fixed before training starts, that assigns each step its own learning rate η_t . The standard plan has two parts — a short *warmup* that ramps η up from near zero, and a long *decay* that brings it back down — and this section explains what each part is for.

11.1 Why one constant η cannot win

What Learning Means established the two-sided trap: too large an η overshoots or diverges, too small an η crawls. Noise sharpens the trap into a trade-off that changes *over time*. Every step is $-\eta g_t$, so the leftover jitter of §10.1 — the part of each step that undoes the last — has amplitude proportional to η : halve η and the wobble halves. This is the noise floor of *What Learning Means*, seen from the mechanics side: at a fixed η the walk descends until progress and jitter are the same size, and then hovers.

But the two halves of training want opposite things. Early on, the parameters are far from good — what matters is covering distance, and a large η with a large floor is fine. Late in training, the walk is *at* its floor — the only way further down is to lower the floor, which means shrinking η . No constant can do both. The resolution is to spend the budget in order: travel first with a big η , settle later with a small one. That is the decay half of the schedule; the warmup half exists for a subtler reason.

11.2 Warmup: small steps while nothing is trustworthy

Warmup is the opening phase in which η_t climbs from near 0 to its peak $\eta_{\max}$, usually linearly over the first T_w steps:

$$\eta_t = \frac{t}{T_w} \eta_{\max} \quad \text{for } t \leq T_w$$

— with $T_w = 1000$ and $\eta_{\max} = 0.001$, step 100 runs at $\eta_{100} = (100/1000) \cdot 0.001 = 0.0001$, a tenth of the peak.

Two things are untrustworthy at step 0. First, the network itself: its parameters are §7’s random draw, so the first gradients are computed on a network that knows nothing — large, erratic, and pointing wherever the random start happens to lean. Full-size steps along them can throw the parameters into bad regions before learning begins: scores pushed deep into ReLU’s negative side (dead neurons, §3.3) or far into saturation. Second, the optimizer’s own statistics: Adam’s m and s (§10.3) are running averages that have seen one or two gradients — dividing by a $\sqrt{s}$ estimated from so little makes early steps erratically scaled, bias correction or not. Warmup keeps the steps small until both the network and the optimizer’s bookkeeping have settled into something worth trusting.

11.3 Decay: settle into the minimum

After the warmup, η_t comes back down. The common shapes: *step decay* (divide η by 10 at fixed milestones), *linear decay* (a straight line to 0), and the current default, *cosine decay* — half a cosine wave from the peak to zero over the planned total of T steps:

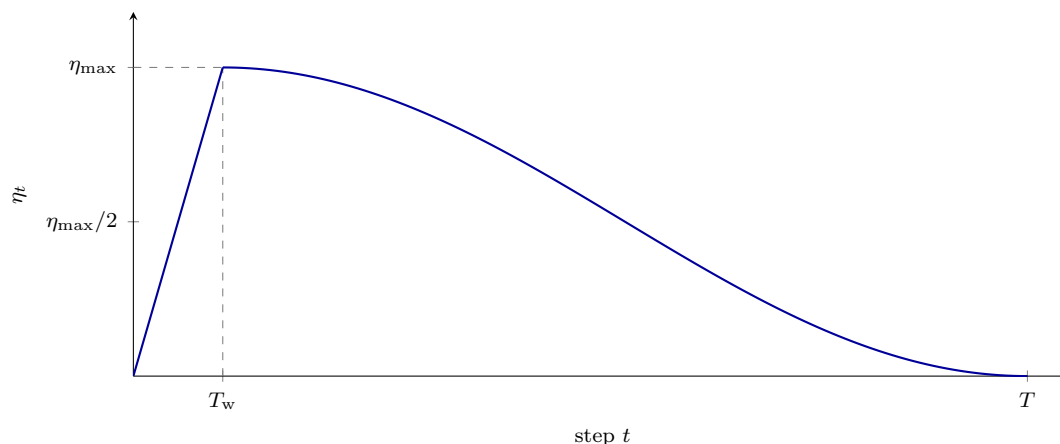
$$\eta_t = \eta_{\max} \cdot \frac{1 + \cos(\pi t/T)}{2}.$$

Three checkpoints show its shape:

$$\begin{aligned}
 t = 0 : \quad & \frac{1 + \cos 0}{2} = \frac{1 + 1}{2} = 1 && \text{(starts at the full } \eta_{\max} \text{)} \\
 t = T/2 : \quad & \frac{1 + \cos(\pi/2)}{2} = \frac{1 + 0}{2} = \frac{1}{2} && \text{(halfway through: half the peak)} \\
 t = T : \quad & \frac{1 + \cos \pi}{2} = \frac{1 - 1}{2} = 0. && \text{(ends at zero)}
 \end{aligned}$$

Despite the name, nothing oscillates: as t runs from 0 to T , the argument $\pi t/T$ runs only from 0 to π — *half* a cosine wave, on which the cosine falls monotonically from 1 to -1 . The schedule descends once and stops; an oscillation would need the argument to pass 2π , that is, t to run on to $2T$ — and training ends at T .

With a warmup in front, the same curve runs on the remaining $T - T_w$ steps. The combined shape is the one every modern training run draws:



the standard schedule: linear warmup to $\eta_{\max}$ over T_w steps, then cosine decay to 0 over the rest

This pair — linear warmup plus cosine decay, driving AdamW (§10.4) — is the training stack of essentially every large language model. Two practical notes. The cosine needs T in advance: the length of training is planned before it starts, not decided along the way. And the schedule varies η but does not choose it — $\eta_{\max}$ is still picked by the person, one of the hyperparameters this chapter’s closing section returns to.

11.4 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
η_t	the learning rate assigned to step t
$\eta_{\max}$	the schedule’s peak learning rate
T_w, T	the warmup length; the planned total number of steps
warmup	the opening ramp from near 0 up to $\eta_{\max}$
cosine decay	$\eta_{\max}(1 + \cos(\pi t/T))/2$: peak to zero, smoothly

12 Gradient clipping

§8.3 ended on an asymmetry: ReLU caps the slope factors and initialization tames the start, but the weight factors have no ceiling — weights can drift large during training, and one unlucky batch can hand the update rule an enormous gradient. One giant step later, the parameters are in NaN country. *Gradient clipping* is the guard built for exactly this: a rule that caps the gradient’s overall size at a threshold c just before the update, while keeping its direction. It is blunt, cheap, and standard.

12.1 The rule

Stack all the parameters’ gradients of one step into a single long vector g . Its overall size is its norm — *Essential Math for AI*’s vector length, $\|g\| = \sqrt{\sum_j g_j^2}$. Clipping by norm compares that length to the chosen threshold c :

$$g_{\text{clip}} = \begin{cases} g, & \|g\| \leq c \quad (\text{a healthy step: nothing happens}) \\ g \cdot \frac{c}{\|g\|}, & \|g\| > c \quad (\text{too long: rescale down to length } c) \end{cases}$$

Multiplying a vector by a positive number keeps its direction and rescales only its length, so the clipped gradient still points exactly where the original pointed — its length is simply pinned at c : $\|g \cdot c/\|g\|\| = (c/\|g\|) \cdot \|g\| = c$.

By hand, with threshold $c = 1$ and a gradient $g = (3, 4)^\top$:

$$\begin{aligned} \|g\| &= \sqrt{g_1^2 + g_2^2} && (\text{the norm, } \textit{Essential Math for AI}) \\ &= \sqrt{3^2 + 4^2} = \sqrt{25} = 5, && (\text{insert } g; \text{ compute}) \\ g_{\text{clip}} &= g \cdot \frac{c}{\|g\|} = (3, 4)^\top \cdot \frac{1}{5} = (0.6, 0.8)^\top, && (5 > c = 1: \text{ rescale by } 1/5) \\ \|g_{\text{clip}}\| &= \sqrt{0.6^2 + 0.8^2} = \sqrt{0.36 + 0.64} = 1 = c. && (\text{length pinned at the cap; direction still } 3 : 4) \end{aligned}$$

A variant, clip-by-value, caps each entry separately instead; it distorts the direction and is rarely preferred — clip-by-norm is the standard.

12.2 Where it sits, and why it still matters

In the loop of §9.1, clipping slots in between the backward pass and the optimizer: backward $\rightarrow$ clip $\rightarrow$ optimizer (§10) $\rightarrow$ update. Momentum and Adam read the *clipped* gradient.

Why is it still needed with ReLU, good initialization, and Adam all in place? Because all of §10’s protections are *averages*, and averages react slowly. Adam’s s blends a new squared gradient in with weight $1 - \beta_2 = 0.001$: a sudden thousandfold spike barely moves $\sqrt{s}$ on the step it arrives, so the spike passes through nearly unscaled — once. Clipping is the opposite kind of defense: instantaneous, per step, no memory. It turns the catastrophic step into an ordinary-sized step *in the same direction* — the spike still says “move that way”, just no longer at ruinous volume. On every healthy step, clipping does nothing at all.

12.3 In practice

The threshold is coarse: $c = 1.0$ is the common default in large-model training, and anywhere in 0.5–5 behaves similarly — the point is to stop the thousandfold spikes of §8.3, not to fine-tune step sizes. It costs one norm computation and one line of code (`clip_grad_norm_` in PyTorch). Real training runs $\log \|g\|$ at every step; spikes in that log are exploding gradients knocking, and the clip absorbing them. Together with AdamW (§10.4) and warmup-plus-cosine (§11.3), clipping completes the standard recipe that large models train under.

12.4 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$\ g\ $	the gradient’s overall length: $\sqrt{\sum_j g_j^2}$
c	the clipping threshold (1.0 typical)
g_{clip}	the gradient after clipping: same direction, length $\leq c$
clip-by-norm	rescale the whole vector when $\ g\ > c$

13 Normalization

§7 chose the *starting* scale with care, and §8 explained what goes wrong when scale drifts. But initialization controls only step 0: as training moves the weights, the pre-activations drift — growing, shrinking, sliding off center — and nothing so far pushes back. *Normalization* is the in-network tool that does: a layer inserted into the network that re-standardizes the numbers passing through it, every step, for the whole run. Its two standard placements — *BatchNorm* and *LayerNorm* — differ only in which direction they average.

13.1 The operation: standardize, then re-dress

Given a collection of numbers $z_1, \dots, z_n$ to normalize, compute their mean and standard deviation — *Essential Math for AI's* pair,

$$\mu = \frac{1}{n} \sum_{i=1}^n z_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (z_i - \mu)^2},$$

and standardize each number: subtract the center, divide by the spread,

$$\hat{z}_i = \frac{z_i - \mu}{\sigma + \varepsilon}$$

(ε the same tiny divide-by-zero guard as in §10.3). The results have mean 0 and spread 1, whatever the inputs were. Then comes the re-dressing: the layer's output is

$$\gamma \hat{z}_i + \beta,$$

where γ (a scale, starting at 1) and β (a shift, starting at 0) are *learned* parameters, one pair per feature. The point of the pair: normalization should remove the *accidental* scale and center — the drift — while leaving the network free to learn any scale and center it actually wants. γ and β are ordinary parameters, trained by the same backpropagation and optimizer as everything else (§6's tape records the normalization like any other operation).

Why this helps, in this chapter's terms: standardized pre-activations sit near 0, which is exactly the healthy zone of §3.3 — σ and $\tanh$ at their steepest, ReLU's inputs straddling the live side — so the slope factors of §8.2 stay alive *during* training, not just at the start. §7 set the scales right at step 0; normalization keeps re-setting them for the rest of the run. In practice this makes training markedly more stable and tolerant of larger learning rates.

13.2 BatchNorm and LayerNorm: which direction you average

The formula needs a *set* of numbers to average over, and the batch matrix of §4.5 offers two directions. Recall its hidden pre-activations for our two examples,

$$Z^{(1)} = \begin{pmatrix} 1 & 2 \\ -1.5 & 3 \end{pmatrix}$$

— one row per example, one column per neuron.

BatchNorm normalizes every *column*: one (μ, σ) per neuron, computed across the batch. Write $Z_{ij}^{(1)}$ for the entry in row i , column j — the entry convention of §5.3 — and $\hat{Z}_{ij}^{\text{BN}}$ for the same

entry after BatchNorm. Column 1, neuron 1's scores on the two examples, holds $Z_{11}^{(1)} = 1$ and $Z_{21}^{(1)} = -1.5$:

$$\begin{aligned}
\mu &= \frac{Z_{11}^{(1)} + Z_{21}^{(1)}}{2} = \frac{1 + (-1.5)}{2} = -0.25, & (\text{the column's mean}) \\
\sigma &= \sqrt{\frac{(Z_{11}^{(1)} - \mu)^2 + (Z_{21}^{(1)} - \mu)^2}{2}} & (\text{the standard deviation}) \\
&= \sqrt{\frac{(1 + 0.25)^2 + (-1.5 + 0.25)^2}{2}} & (\text{insert the column and } \mu) \\
&= \sqrt{\frac{1.5625 + 1.5625}{2}} = 1.25, & (\text{square; average; take the root}) \\
(\hat{Z}_{11}^{\text{BN}}, \hat{Z}_{21}^{\text{BN}}) &= \left(\frac{1 + 0.25}{1.25}, \frac{-1.5 + 0.25}{1.25} \right) = (1, -1). & (\text{subtract } \mu; \text{ divide by } \sigma)
\end{aligned}$$

Column 2 gets the same treatment with its own pair, using $Z_{12}^{(1)} = 2$ and $Z_{22}^{(1)} = 3$:

$$\begin{aligned}
\mu &= \frac{Z_{12}^{(1)} + Z_{22}^{(1)}}{2} = \frac{2 + 3}{2} = 2.5, & (\text{column 2's mean}) \\
\sigma &= \sqrt{\frac{(2 - 2.5)^2 + (3 - 2.5)^2}{2}} = \sqrt{\frac{0.25 + 0.25}{2}} = 0.5, & (\text{its standard deviation}) \\
(\hat{Z}_{12}^{\text{BN}}, \hat{Z}_{22}^{\text{BN}}) &= \left(\frac{2 - 2.5}{0.5}, \frac{3 - 2.5}{0.5} \right) = (-1, 1). & (\text{subtract } \mu; \text{ divide by } \sigma)
\end{aligned}$$

Every entry has now been used once, and each result returns to the cell it came from — normalization rewrites values, never positions:

$$Z_{\text{BN}}^{(1)} = \begin{pmatrix} \hat{Z}_{11}^{\text{BN}} & \hat{Z}_{12}^{\text{BN}} \\ \hat{Z}_{21}^{\text{BN}} & \hat{Z}_{22}^{\text{BN}} \end{pmatrix} = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

LayerNorm normalizes every *row*: one (μ, σ) per example, computed across the layer's neurons; write $\hat{Z}_{ij}^{\text{LN}}$ for an entry after LayerNorm. Row 1 holds $Z_{11}^{(1)} = 1$ and $Z_{12}^{(1)} = 2$:

$$\begin{aligned}
\mu &= \frac{Z_{11}^{(1)} + Z_{12}^{(1)}}{2} = \frac{1 + 2}{2} = 1.5, & (\text{the row's mean}) \\
\sigma &= \sqrt{\frac{(Z_{11}^{(1)} - \mu)^2 + (Z_{12}^{(1)} - \mu)^2}{2}} & (\text{same formula, along the row}) \\
&= \sqrt{\frac{(1 - 1.5)^2 + (2 - 1.5)^2}{2}} & (\text{insert the row and } \mu) \\
&= \sqrt{\frac{0.25 + 0.25}{2}} = 0.5, & (\text{square; average; take the root}) \\
(\hat{Z}_{11}^{\text{LN}}, \hat{Z}_{12}^{\text{LN}}) &= \left(\frac{1 - 1.5}{0.5}, \frac{2 - 1.5}{0.5} \right) & (\text{subtract } \mu; \text{ divide by } \sigma) \\
&= (-1, 1), & (\text{evaluate})
\end{aligned}$$

and row 2, with $Z_{21}^{(1)} = -1.5$ and $Z_{22}^{(1)} = 3$:

$$\begin{aligned}\mu &= \frac{-1.5 + 3}{2} = 0.75, & (\text{row 2's mean}) \\ \sigma &= \sqrt{\frac{(-1.5 - 0.75)^2 + (3 - 0.75)^2}{2}} & (\text{its standard deviation}) \\ &= \sqrt{\frac{5.0625 + 5.0625}{2}} = 2.25, & (\text{square; average; take the root}) \\ (\hat{Z}_{21}^{\text{LN}}, \hat{Z}_{22}^{\text{LN}}) &= \left(\frac{-1.5 - 0.75}{2.25}, \frac{3 - 0.75}{2.25} \right) = (-1, 1), & (\text{subtract } \mu; \text{ divide by } \sigma)\end{aligned}$$

giving

$$Z_{\text{LN}}^{(1)} = \begin{pmatrix} \hat{Z}_{11}^{\text{LN}} & \hat{Z}_{12}^{\text{LN}} \\ \hat{Z}_{21}^{\text{LN}} & \hat{Z}_{22}^{\text{LN}} \end{pmatrix} = \begin{pmatrix} -1 & 1 \\ -1 & 1 \end{pmatrix}.$$

(Every pair lands on ± 1 — the quirk of standardizing exactly two numbers, which always gives ± 1 . At realistic sizes the values keep their shape; only center and scale are removed.)

Two things to read off the pair of results. First, a network uses *one* method per normalization layer: BatchNorm normalizes every column, or LayerNorm every row — never a mix within one layer. Second, the choice is visible in the outcome: the same $Z^{(1)}$ produced two *different* matrices — cell $(1, 1)$ became 1 under BatchNorm but -1 under LayerNorm — because each method measures an entry against a different peer group.

Where do these numbers go? They *replace* the originals: each value passes through the re-dressing $\gamma\hat{z} + \beta$ — at the start $\gamma = 1, \beta = 0$, unchanged — and continues down the network on the path the raw pre-activation would have taken. Suppose the layer uses BatchNorm. The matrix that reaches the ReLU is $Z_{\text{BN}}^{(1)}$, so

$$H = \text{ReLU}(Z_{\text{BN}}^{(1)}) = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

This H is the batch's hidden-activation matrix: it takes the place of §4.5's $H = \begin{pmatrix} 1 & 2 \\ 0 & 3 \end{pmatrix}$, and the output layer reads it exactly as before — $Z^{(2)} = HW_2^\top + b_2$, and onward. Nothing downstream changes its shape or its procedure; only the numbers flowing through are tamer.

The choice of direction is not cosmetic. BatchNorm's statistics depend on the *batchmates*: the same example normalizes differently in a different batch, and at test time there may be no batch at all — so BatchNorm keeps running averages of μ and σ during training and freezes them for inference, a wrinkle that also makes it unreliable at small B . LayerNorm's statistics live inside the single example: no batch dependence, identical behavior in training and inference, works at $B = 1$. That robustness is why transformers — and with them all large language models — use LayerNorm; BatchNorm remains common in vision networks.

	BatchNorm	LayerNorm
averages over	the batch (one column = one neuron)	the layer (one row = one example)
depends on batchmates	yes	no
training vs. inference	running averages, then frozen	identical
typical home	vision networks	transformers, LLMs

13.3 Where it sits in the network

Normalization is inserted as a layer of its own, most commonly between the linear part and the activation: $z = Wx + b$, then normalize, then γ, β , then φ . (Where exactly to place it has variants — transformers, for instance, normalize *before* their sublayers in most modern designs — but every variant is one line of code: `BatchNorm1d` or `LayerNorm` in PyTorch.) To autograd it is just more recorded operations; to the optimizer, γ and β are just more parameters.

13.4 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
μ, σ	mean and standard deviation of the normalized set
$\hat{z}_i$	the standardized value $(z_i - \mu)/(\sigma + \varepsilon)$
$\hat{Z}_{ij}^{\text{BN}}, \hat{Z}_{ij}^{\text{LN}}$	entry (i, j) after BatchNorm / after LayerNorm (different numbers)
γ, β	the learned re-dressing: scale (starts 1) and shift (starts 0)
BatchNorm	standardize each neuron across the batch
LayerNorm	standardize each example across the layer

14 Dropout

Everything so far makes training *work*; *What Learning Means* spent three sections on what working training can still do wrong — overfit — and on the regularizers that push back. *Dropout* is the regularizer native to neural networks: during training, each neuron’s output is silenced — set to 0 — at random, independently at every step, with a fixed probability p ; at inference, nothing is silenced. This section gives the exact rule, the reason the survivors are rescaled, and why deliberately damaging the network helps it generalize.

14.1 The rule

At each training step, each neuron draws a *mask* value

$$m_i = \begin{cases} 0 & \text{with probability } p \quad (\text{dropped: this neuron is silent this step}) \\ 1 & \text{with probability } 1 - p \quad (\text{kept}) \end{cases}$$

and its activation h_i is replaced by

$$h'_i = \frac{m_i h_i}{1 - p}.$$

The division is the *inverted-dropout* rescale: the survivors speak louder so that, on average, the layer delivers what it always delivered. The average is over the random mask, with *Essential Math for AI*’s expectation:

$$\begin{aligned} \mathbb{E}[h'_i] &= \mathbb{E}[m_i] \cdot \frac{h_i}{1 - p} && (h_i \text{ is fixed this step; only } m_i \text{ is random}) \\ &= (0 \cdot p + 1 \cdot (1 - p)) \cdot \frac{h_i}{1 - p} && (\text{the expectation of the mask, from the definition}) \\ &= (1 - p) \cdot \frac{h_i}{1 - p} = h_i. && (\text{the } 1 - p \text{ cancels}) \end{aligned}$$

So the typical size of the signal is preserved — the scale hygiene of §7 and §13 is not disturbed.

By hand, on the running network’s hidden activations $h = (1, 2)^\top$ (§4.2), with $p = 0.5$. Suppose this step’s draw is $m = (0, 1)$:

$$h' = \left(\frac{0 \cdot 1}{0.5}, \frac{1 \cdot 2}{0.5} \right) = (0, 4);$$

neuron 1 is silent, neuron 2 speaks at double volume. The next step draws a fresh mask — perhaps $m = (1, 0)$, giving $h' = (2, 0)$ — so every step trains a *different* randomly chosen sub-network. At inference the mask is gone entirely: $h' = h$, no drops, no rescale. (The train/inference switch is the same one §13.2 met with BatchNorm’s running averages; frameworks flip both with one `train()/eval()` toggle.)

14.2 Why silencing helps

Dropout is a regularizer, and both of its effects are stories *What Learning Means* has already told in general form.

It forbids conspiracies. Without dropout, two neurons can settle into a brittle pair — one’s quirk canceling the other’s — useful only together, which is exactly the kind of intricate, noise-fitting

structure overfitting is made of. Under dropout no neuron can rely on a specific partner being awake, so each feature is pushed to be useful *on its own*, and the network builds redundancy instead of conspiracies.

It trains an average. Each step trains a different sub-network; the full network at inference acts like an average over the enormous family of sub-networks that were trained. And averaging is variance medicine — *What Learning Means* showed averaging two overfit curves canceling much of their wiggle. Dropout buys that averaging without ever training the family separately.

In practice p runs from 0.1 to 0.5, largest in big fully-connected layers where overfitting bites hardest. Transformers use light dropout ($p \approx 0.1$) or, when the data is enormous, sometimes none — the more data, the less the regularizer is needed, exactly as *What Learning Means* would predict.

14.3 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
p	the drop probability (0.1–0.5 typical)
m_i	the mask: 0 (dropped) with probability p , else 1
h'_i	the activation after dropout: $m_i h_i / (1 - p)$
inverted dropout	rescaling survivors by $1/(1 - p)$ so the average is unchanged
sub-network	the network with this step's dropped neurons removed

15 Parameters and hyperparameters

This chapter has introduced a small crowd of numbers, and they split cleanly into two kinds. A *parameter* is a number that the training loop itself adjusts: it receives a gradient from backpropagation and is moved by the optimizer. A *hyperparameter* is a number (or a choice) fixed by the person, outside the loop: it shapes how training runs but receives no gradient. The test is mechanical — does `.grad` exist for it and does the update rule touch it? Yes: parameter. No: hyperparameter. This closing section sorts everything the chapter built into the two bins.

15.1 The inventory

The parameter bin is short — training adjusts exactly two families of numbers:

Parameters	What they are	Where
W_1, b_1, W_2, b_2	the layers' weights and biases (9 in the running network)	§4.2
γ, β	normalization's learned scale and shift	§13.1

(A third, smaller family lives beside them: the optimizer's own bookkeeping — momentum's v , Adam's m and s , BatchNorm's running averages. These are neither chosen by the person nor part of the model's answer; they are training-time state, kept while the loop runs and not needed to use the trained network.)

The hyperparameter bin is where the rest of the chapter lives:

Hyperparameter	What it sets	Where
m, D	width and depth: the network's shape	§4.1 , §4.4
φ, ψ	the activation functions	§3.4
Xavier / He	the initial weight scale	§7.3
optimizer; $\beta_1, \beta_2, \varepsilon$	how gradients become steps	§10
$\eta_{\max}, T_w, T, \text{shape}$	the learning-rate schedule	§11
B	the batch size	§9.3
epochs	how long training runs	§9.2
c	the gradient-clipping threshold	§12
BatchNorm / LayerNorm	the normalization direction	§13.2
p	the dropout rate	§14
λ	weight decay's strength (AdamW)	§10.4

The split explains the vocabulary of practice: “the model has a billion parameters” counts the first bin; “we tuned the hyperparameters” means the second was searched by hand.

15.2 How hyperparameters get chosen

Not by gradients — none exist for them. Some are not even numbers (ReLU versus tanh); others are integers or whole schedules, with nothing to differentiate. The honest procedure is the one *What Learning Means* built the validation set for: train with a candidate setting, score the result on validation data the training never saw, keep the best candidate, and spend the test set exactly once at the very end. (Searching too many candidates against the same validation set re-imports the leak that chapter warned about — the score of the winner flatters it.)

In practice the search is far smaller than the table suggests, because the defaults this chapter named carry most of the weight: ReLU, He initialization, AdamW with $\beta_1 = 0.9$ and $\beta_2 = 0.999$, warmup plus cosine, $c = 1$, light dropout. What usually needs real tuning is short: $\eta_{\max}$ above all — the

learning rate remains *the* hyperparameter — then the batch size (often just “as large as memory allows”), and the width and depth, which are set by compute budget as much as by search. With that, the mechanics of this chapter are complete: a network, trained end to end, every number accounted for. How these layers get *arranged* — into convolutional networks, into transformers — is the next chapter’s subject.

15.3 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
parameter	adjusted by the loop: has a gradient, moved by the optimizer
hyperparameter	fixed by the person: shapes training, receives no gradient
optimizer state	the loop’s bookkeeping (v ; m , s ; running averages)